

Contents

1 Basic	1	6.11 Factorial without prime factor*	15
1.1 Hash.sh	1	6.12 QuadraticResidue*	15
1.2 readchar	1	6.13 PiCount*	15
1.3 Black Magic	1	6.14 Discrete Log*	15
1.4 Pragma Optimization	1	6.15 Berlekamp Massey	15
1.5 Sanitize Fix	1	6.16 Primes	16
		6.17 Theorem	16
		6.18 Estimation	16
		6.19 Euclidean Algorithms	16
		6.20 General Purpose Numbers	16
		6.21 Tips for Generating Functions	17
2 Graph	1	7 Polynomial	17
2.1 EBCC*	1	7.1 Fast Fourier Transform	17
2.2 VBCC*	1	7.2 FFT Arbitrary Mod	17
2.3 SCC*	2	7.3 Number Theory Transform*	17
2.4 ZSAT*	2	7.4 NTT Primes	17
2.5 MinimumMeanCycle*	2	7.5 Fast Walsh Transform*	18
2.6 Virtual Tree*	2	7.6 Polynomial Operation	18
2.7 Maximum Clique Dyn*	2	7.7 Value Polynomial	19
2.8 Minimum Steiner Tree*	3	7.8 Newton's Method	19
2.9 Dominator Tree*	3		
2.10 Minimum Arborescence*	3	8 Geometry	20
2.11 Vizing's theorem*	4	8.1 Default Code	20
2.12 Minimum Clique Cover*	4	8.2 PointSegDist*	20
2.13 NumberOfMaximalClique*	4	8.3 Heart	20
		8.4 point in circle	20
3 Data Structure	4	8.5 Convex hull*	20
3.1 Discrete Trick	4	8.6 PointInConvex*	20
3.2 Mergeable Heap	5	8.7 TangentPointToHull*	20
3.3 BIT kth*	5	8.8 Intersection of line and convex	20
3.4 Interval Container*	5	8.9 minMaxEnclosingRectangle*	21
3.5 Heavy light Decomposition*	5	8.10 VectorInPoly*	21
3.6 Centroid Decomposition*	5	8.11 PolyUnion*	21
3.7 Treap*	6	8.12 Polar Angle Sort*	21
3.8 LiChaoST*	6	8.13 Half plane intersection*	21
3.9 Link cut tree*	6	8.14 HPI Alternative Form	21
3.10 KDTree	7	8.15 RotatingSweepLine	22
		8.16 Minimum Enclosing Circle*	22
4 Flow/Matching	7	8.17 Intersection of two circles*	22
4.1 Bipartite Matching*	7	8.18 Intersection of polygon and circle*	22
4.2 Kuhn Munkres*	7	8.19 Intersection of line and circle*	22
4.3 MincostMaxFlow*	8	8.20 Tangent line of two circles	22
4.4 Maximum Simple Graph Matching*	8	8.21 CircleCover*	22
4.5 Maximum Weight Matching*	9	8.22 3Dpoint*	23
4.6 SW-mincut	10	8.23 Convexhull3D*	23
4.7 BoundedFlow*(Dinic*)	10	8.24 DelaunayTriangulation*	24
4.8 Gomory Hu tree*	10	8.25 Triangulation Voronoi*	24
4.9 Minimum Cost Circulation*	10	8.26 Minkowski Sum*	24
4.10 Flow Models	11	8.27 Closest Pair	24
5 String	11	9 Else	24
5.1 KMP	11	9.1 Cyclic Ternary Search*	24
5.2 Z-value*	11	9.2 Mo's Algorithm(With modification)	24
5.3 Manacher*	11	9.3 Mo's Algorithm On Tree	24
5.4 Suffix Array*	11	9.4 Additional Mo's Algorithm Trick	25
5.5 SAIS C++20*	12	9.5 Hilbert Curve	25
5.6 Aho-Corasick Automatan*	12	9.6 DynamicConvexTrick*	25
5.7 Smallest Rotation*	12	9.7 All LCS*	25
5.8 De Bruijn sequence*	12	9.8 DLX*	25
5.9 Extended SAM*	12	9.9 Matroid Intersection	26
5.10 PalTree*	13	9.10 Matroid Intersection	26
5.11 Main Lorentz	13	9.11 AdaptiveSimpson*	26
		9.12 Simulated Annealing	26
6 Math	13	9.13 Tree Hash*	27
6.1 ax+by=gcd(only exgcd *)	13	9.14 Binary Search On Fraction*	27
6.2 Floor and Ceil	13	9.15 1D1D	27
6.3 Floor Enumeration	13	9.16 SMAWK	27
6.4 Mod Min	13	9.17 Min Plus Convolution*	27
6.5 Gaussian integer gcd	13	9.18 Bitset LCS	27
6.6 Miller Rabin*	14	9.19 Python	28
6.7 System of Linear Equations	14		
6.8 Pollard Rho*	14		
6.9 Simplex Algorithm	14		
6.9.1 Construction	14		
6.10 chineseRemainder	15		

1 Basic

1.1 Hash.sh

```
echo "Useful for verifying that there aren't mistypes."
cpp $1 -dD -P
-fpreprocessed | tr -d '[:space:]' | md5sum | cut -c-6
```

1.2 readchar [a419b9]

```
inline char readchar() {
    static const size_t bufsize = 65536;
    static char buf[bufsize];
    static char *p = buf, *end = buf;
    if (p == end) end = buf
        + fread_unlocked(buf, 1, bufsize, stdin), p = buf;
    return *p++;
}
```

1.3 Black Magic [afb343]

```
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/assoc_container.hpp> // rb_tree
#include <ext/rope> // rope
using namespace __gnu_pbds;
```

```
using namespace __gnu_cxx; // rope
typedef __gnu_pbds::priority_queue<int> heap;
int main() {
    heap h1, h2; // max heap
    h1.push(1), h1.push(3), h2.push(2), h2.push(4);
    h1.join(h2); // h1 = {1, 2, 3, 4}, h2 = {};
    tree<ll, null_type, less<ll>,
        rb_tree_tag, tree_order_statistics_node_update> st;
    tree<ll, ll, less<ll>,
        rb_tree_tag, tree_order_statistics_node_update> mp;
    for (int x : {0, 3, 20, 50}) st.insert(x);
    assert(
        st.order_of_key(3) == 1 && st.order_of_key(4) == 2);
    assert(*st.
        find_by_order(2) == 20 && *st.lower_bound(4) == 20);
    rope<char> *root[10]; // nsqrt(n)
    root[0] = new rope<char>();
    root[1] = new rope<char>(*root[0]);
    // root[1]->insert(pos, 'a');
    // root[1]->at(pos); 0-base
    // root[1]->erase(pos, size);
}
// __int128_t, __float128_t
// for (int i = bs
    ._Find_first(); i < bs.size(); i = bs._Find_next(i));
```

1.4 Pragma Optimization [49e509]

```
#pragma GCC optimize("Ofast,no-stack-protector")
#pragma GCC optimize("no-math-errno,unroll-loops")
#pragma GCC target("sse,sse2,sse3,ssse3,sse4")
#pragma GCC target("popcnt,abm,mmx,avx,arch=skylake")
__builtin_ia32_ldmxcsr(__builtin_ia32_stmxcsr())|0x8040
```

1.5 Sanitize Fix [6bec31]

```
#include <sys/personality.h>
#include <bits/stdc++.h>
int main(int argc, char** argv){
    personality(ADDR_NO_RANDOMIZE);
    execv(argv[1],argv+1);
}
```

2 Graph

2.1 EBCC* [8ab181]

```
struct EBCC { // need adj
    int n, dfcnt = 0, bccnt = 0;
    vector<int> dfn, low, bcc, st;
    EBCC() {}
    EBCC(int n)
        : n(n) { dfn = low = bcc = vector<int>(n + 1, 0); }
    void tarjan(int pos, int fa) {
        dfn[pos] = low[pos] = ++dfcnt;
        st.pb(pos);
        bool vs = 0;
        for (int np : adj[pos]) {
            if (np != fa || vs) {
                if (dfn[np] == 0) {
                    tarjan(np, pos);
                    low[pos] = min(low[pos], low[np]);
                    if (dfn[pos] < low[np]) {
                        bccnt++;
                        while (1) {
                            int x = st.back();
                            bcc[x] = bccnt;
                            st.pop_back();
                            if (x == np) break;
                        }
                    } else { low[pos] = min(low[pos], dfn[np]); }
                } else { vs = 1; }
            }
        }
        if (pos == fa) {
            bccnt++;
            while (st.size()) {
                bcc[st.back()] = bccnt;
                st.pop_back();
            }
        }
    }
    void work() {
        FOR (i, 1, n) if (dfn[i] == 0) tarjan(i, i);
    }
};
```

2.2 VBCC* [8fadbe]

```

struct VBCC { // need adj
    int n, dfcnt = 0, bccnt = 0;
    vector<int> dfn, low, st;
    vector<vector<int>> bcc;
    VBCC() {}
    VBCC(int n) : n(n) {
        dfn = low = vector<int>(n + 1, 0);
        bcc = vector<vector<int>>(n + 1, vector<int>());
    }
    void tarjan(int pos, int fa) {
        dfn[pos] = low[pos] = ++dfcnt;
        st.pb(pos);
        for (int np : adj[pos]) {
            if (np != fa) {
                if (dfn[np] == 0) {
                    tarjan(np, pos);
                    low[pos] = min(low[pos], low[np]);
                    if (dfn[pos] <= low[np]) {
                        bccnt++;
                        while (1) {
                            int x = st.back();
                            bcc[x].pb(bccnt);
                            st.pop_back();
                            if (x == np) break;
                        }
                        bcc[pos].pb(bccnt);
                    }
                } else { low[pos] = min(low[pos], dfn[np]); }
            }
        }
        if (pos == fa) {
            st.pop_back();
            if (bcc[pos].size() == 0) bcc[pos].pb(++bccnt);
        }
    }
    void work() {
        FOR (i, 1, n) if (dfn[i] == 0) tarjan(i, i);
    }
};

```

2.3 SCC* [7c6db3]

```

struct SCC { // need adj
    int n, dfcnt = 0, sccnt = 0;
    vector<int> dfn, scc, st;
    vector<vector<int>> scc_adj;
    SCC() {}
    SCC(int n) : n(n), dfn(n + 1), scc(n + 1), scc_adj(n + 1) {}
    void tarjan(int u) { // scc[] numbered sink first
        int lw = dfn[u] = ++dfcnt, x;
        st.pb(u);
        for (int v : adj[u]) if (!scc[v]) {
            if (!dfn[v]) tarjan(v);
            lw = min(lw, dfn[v]);
        }
        if (lw == dfn[u]) {
            sccnt++;
            do {
                x = st.back(), st.pop_back(), scc[x] = sccnt;
            } while (x != u);
        }
        dfn[u] = lw; // dfn[] now holds the low-link
    }
    void work() {
        FOR (i, 1, n) if (!dfn[i]) tarjan(i);
    }
    void build_adj() {
        FOR (i, 1, n) for (int j : adj[i])
            if (scc[i] != scc[j])
                scc_adj[scc[i]].pb(scc[j]);
    }
};

```

2.4 2SAT* [11893d]

```

// 0-base, reuses SCC, so it needs the same adj.
// ~x is NOT x; literal a lives on node en(a) + 1
struct TwoSat {
    int n;
    vector<int> ans; // 0 / 1, set by solve()
    TwoSat(int n = 0) : n(n) {}
    int en(int a) { return max(a * 2, -1 - a * 2); }
    int add_var() { return n++; } // adj must have room
    void either(int a, int b) { // a or b
        int x = en(a), y = en(b);
        adj[(x ^ 1) + 1].pb(y + 1);
        adj[(y ^ 1) + 1].pb(x + 1);
    }
};

```

```

void set_value(int x) { either(x, x); }
void at_most_one(const vector<int> &v) {
    if (v.size() <= 1) return;
    int cur = ~v[0];
    FOR (i, 2, (int)v.size() - 1) {
        int nxt = add_var();
        either(cur, ~v[i]), either(cur, nxt);
        either(~v[i], nxt), cur = ~nxt;
    }
    either(cur, ~v[1]);
}
bool solve() { // scc[] is numbered sink first
    SCC scc(n * 2);
    scc.work();
    ans.assign(n, 0);
    FOR (i, 0, n - 1) {
        int a = i * 2 + 1, b = i * 2 + 2;
        if (scc.scc[a] == scc.scc[b]) return false;
        ans[i] = scc.scc[a] < scc.scc[b];
    }
    return true;
}
};

```

2.5 MinimumMeanCycle* [37db7a]

```

typedef pair<ll, ll> pll;
ll road[N][N]; // input here
struct MinimumMeanCycle {
    ll dp[N + 5][N], n;
    pll solve() {
        ll a = -1, b = -1, L = n + 1;
        FOR (i, 2, L)
            FOR (k, 0, n - 1)
                FOR (j, 0, n - 1)
                    dp[i][j] =
                        min(dp[i - 1][k] + road[k][j], dp[i][j]);
        FOR (i, 0, n - 1) {
            if (dp[L][i] >= INF) continue;
            ll ta = 0, tb = 1;
            FOR (j, 1, n - 1)
                if (dp[j][i] < INF &&
                    ta * (L - j) < (dp[L][i] - dp[j][i]) * tb)
                    ta = dp[L][i] - dp[j][i], tb = L - j;
            if (ta == 0) continue;
            if (a == -1 || a * tb > ta * b) a = ta, b = tb;
        }
        if (a != -1) {
            ll g = __gcd(a, b);
            return pll(a / g, b / g);
        }
        return pll(-1LL, -1LL);
    }
    void init(int _n) {
        n = _n;
        FOR (i, 0, n - 1)
            FOR (j, 0, n - 1) dp[i + 2][j] = INF;
    }
};

```

2.6 Virtual Tree* [6e8a09]

```

vector<int> vG[N];
int top, st[N];
void insert(int u) {
    if (top == -1) return st[++top] = u, void();
    int p = LCA(st[top], u);
    if (p == st[top]) return st[++top] = u, void();
    while (top >= 1 && dep[st[top - 1]] >= dep[p])
        vG[st[top - 1]].pb(st[top]), --top;
    if (st[top] != p)
        vG[p].pb(st[top]), --top, st[++top] = p;
    st[++top] = u;
}
void reset(int u) {
    for (int i : vG[u]) reset(i);
    vG[u].clear();
}
void solve(vector<int> &v) {
    top = -1;
    sort(v.begin(), v.end(),
        [&](int a, int b) { return dfn[a] < dfn[b]; });
    for (int i : v) insert(i);
    while (top > 0) vG[st[top - 1]].pb(st[top]), --top;
    // do something
    reset(v[0]);
}

```

2.7 Maximum Clique Dyn* [3bcbd8]

```

struct MaxClique { // fast when N <= 100
    bitset<N> G[N], cs[N];
    int ans, sol[N], q, cur[N], d[N], n;
    void init(int _n) {
        n = _n;
        FOR (i, 0, n - 1) G[i].reset();
    }
    void add_edge(int u, int v) {
        G[u][v] = G[v][u] = 1;
    }
    void pre_dfs(vector<int> &r, int l, bitset<N> mask) {
        if (l < 4) {
            for (int i : r) d[i] = (G[i] & mask).count();
            sort(r.begin(), r.end(),
                [&](int x, int y) { return d[x] > d[y]; });
        }
        vector<int> c((int)r.size());
        int lft = max(ans - q + 1, 1), rgt = 1, tp = 0;
        cs[1].reset(), cs[2].reset();
        for (int p : r) {
            int k = 1;
            while ((cs[k] & G[p]).any()) ++k;
            if (k > rgt) cs[++rgt + 1].reset();
            cs[k][p] = 1;
            if (k < lft) r[tp++] = p;
        }
        FOR (k, lft, rgt)
            for (int p = cs[k]
                .Find_first(); p < N; p = cs[k].Find_next(p))
                r[tp] = p, c[tp] = k, ++tp;
        dfs(r, c, l + 1, mask);
    }
    void dfs(vector<int> &r, vector<int> &c, int l, bitset<N> mask) {
        while (!r.empty()) {
            int p = r.back();
            r.pop_back(), mask[p] = 0;
            if (q + c.back() <= ans) return;
            cur[q++] = p;
            vector<int> nr;
            for (int i : r) if (G[p][i]) nr.pb(i);
            if (!nr.empty()) pre_dfs(nr, l, mask & G[p]);
            else if (q > ans) ans = q, copy_n(cur, q, sol);
            c.pop_back(), --q;
        }
    }
    int solve() {
        vector<int> r(n);
        ans = q = 0, iota(r.begin(), r.end(), 0);
        pre_dfs(r, 0, bitset<N>(string(n, '1')));
        return ans;
    }
};

```

2.8 Minimum Steiner Tree* [9a9a9d]

```

struct SteinerTree { // 0-base, need INF
    int n; // W <= 1e9 and V <= 100 already overflow int
    ll dst[N][N], dp[1 << T][N], tdst[N];
    ll vcst[N]; // the cost of vertices
    void init(int _n) {
        n = _n;
        FOR (i, 0, n - 1) {
            fill_n(dst[i], n, INF);
            dst[i][i] = vcst[i] = 0;
        }
    }
    void chmin(ll &x, ll val) {
        x = min(x, val);
    }
    void add_edge(int ui, int vi, ll wi) {
        chmin(dst[ui][vi], wi);
    }
    void shortest_path() { // dst[i][j] ends up counting
        FOR (k, 0, n - 1) // every vcst on i -> j but i's
            FOR (i, 0, n - 1)
                FOR (j, 0, n - 1)
                    chmin(
                        dst[i][j], dst[i][k] + vcst[k] + dst[k][j]);
        FOR (i, 0, n - 1) FOR (j, 0, n - 1)
            if (i != j) dst[i][j] += vcst[j];
    }
    ll solve(const vector<int> &ter) {
        shortest_path();
        int t = (int)ter.size(), full = (1 << t) - 1;
        FOR (i, 0, full) fill_n(dp[i], n, INF);
        copy_n(vcast, n, dp[0]);
        FOR (msk, 1, full) {
            if (!(msk & (msk - 1))) {

```

```

                int who = __lg(msk);
                FOR (i, 0, n - 1)
                    dp[msk][i] = vcst[ter[who]] + dst[ter[who]][i];
            }
            FOR (i, 0, n - 1)
                for (int sub
                    = (msk - 1) & msk; sub; sub = (sub - 1) & msk)
                    chmin(dp[msk][i], dp[sub][i] + dp[msk ^ sub][i] - vcst[i]);
            FOR (i, 0, n - 1) {
                tdst[i] = INF;
                FOR (j, 0, n - 1)
                    chmin(tdst[i], dp[msk][j] + dst[j][i]);
            }
            copy_n(tdst, n, dp[msk]);
        }
        return *min_element(dp[full], dp[full] + n);
    }
}; // O(V 3^T + V^2 2^T)

```

2.9 Dominator Tree* [68d093]

```

struct dominator_tree { // 1-base
    vector<int> G[N], rG[N];
    int n, pa[N], dfn[N], id[N], Time;
    int semi[N], idom[N], best[N];
    vector<int> tree[N]; // dominator_tree
    void init(int _n) {
        n = _n;
        FOR (i, 1, n) G[i].clear(), rG[i].clear();
    }
    void add_edge(int u, int v) {
        G[u].pb(v), rG[v].pb(u);
    }
    void dfs(int u) {
        id[dfn[u] = ++Time] = u;
        for (auto v : G[u])
            if (!dfn[v]) dfs(v), pa[dfn[v]] = dfn[u];
    }
    int find(int y, int x) {
        if (y <= x) return y;
        int tmp = find(pa[y], x);
        if (semi[best[y]] > semi[best[pa[y]]])
            best[y] = best[pa[y]];
        return pa[y] = tmp;
    }
    void tarjan(int root) {
        Time = 0;
        FOR (i, 1, n) {
            dfn[i] = idom[i] = 0;
            tree[i].clear();
            best[i] = semi[i] = i;
        }
        dfs(root);
        for (int i = Time; i > 1; i--) {
            int u = id[i];
            for (auto v : rG[u])
                if (v = dfn[v]) {
                    find(v, i);
                    semi[i] = min(semi[i], semi[best[v]]);
                }
            tree[semi[i]].pb(i);
            for (auto v : tree[pa[i]]) {
                find(v, pa[i]);
                idom[v] =
                    semi[best[v]] == pa[i] ? pa[i] : best[v];
            }
            tree[pa[i]].clear();
        }
        FOR (i, 2, Time) {
            if (idom[i] != semi[i]) idom[i] = idom[idom[i]];
            tree[id[idom[i]]].pb(id[i]);
        }
    }
};

```

2.10 Minimum Arborescence* [ff1e5a]

```

/* TODO
DSU: disjoint set
- DSU(n), .boss(x), .Union(x, y)
min_heap
<T, Info>: min heap for type {T, Info} with lazy tag
- .push({w, i})
, .top(), .join(heap), .pop(), .empty(), .add_lazy(v)
*/
struct E { int s, t; ll w; }; // 0-base
vector<int> dmst(const vector<E> &e, int n, int root) {
    vector<min_heap<ll, int>> h(n * 2);

```

```

FOR (i, 0, (int)e.size() - 1)
    h[e[i].t].push({e[i].w, i});
DSU dsu(n * 2);
vector<int> v(n * 2, -1), pa(n * 2, -1), r(n * 2);
v[root] = n + 1;
int pc = n;
FOR (i, 0, n - 1)
    if (v[i] == -1) {
        for (int p = i; v[p] == -1 || v[p] == i;
            p = dsu.boss(e[r[p]].s)) {
            if (v[p] == i) {
                int q = p;
                p = pc++;
                do {
                    h[q].add_lazy(-h[q].top().F);
                    pa[q] = p, dsu.Union(p, q),
                    h[p].join(h[q]);
                } while ((q = dsu.boss(e[r[q]].s)) != p);
            }
            v[p] = i;
            while (!h[p].empty() &&
                dsu.boss(e[h[p].top().S].s) == p)
                h[p].pop();
            if (h[p].empty()) return {}; // no solution
            r[p] = h[p].top().S;
        }
    }
vector<int> ans;
for (int i
    = pc - 1; i >= 0; i--) if (i != root && v[i] != n) {
    for (int f = e[r[i]].t; ~f && v[f] != n; f = pa[f])
        v[f] = n;
    ans.pb(r[i]);
}
return ans; // default minimize, returns edgeid array
} // O(Ef(E)), f(E) from min_heap

```

2.11 Vizing's theorem* [4a6f64]

```

#define SZ(a) ((int)a.size())
namespace vizing { //
    returns edge coloring in adjacent matrix G. 1 - based
const int N = 105;
int C[N][N], G[N][N], X[N], vst[N], n;
void init(int _n) { n = _n;
    FOR (i, 0, n)
        FOR (j, 0, n) C[i][j] = G[i][j] = 0;
}
void solve(vector<pair<int, int>> &E) {
    auto update = [&](int u)
    { for (X[u] = 1; C[u][X[u]]; ++X[u]); };
    auto color = [&](int u, int v, int c) {
        int p = G[u][v];
        G[u][v] = G[v][u] = c;
        C[u][c] = v, C[v][c] = u;
        C[u][p] = C[v][p] = 0;
        if (p) X[u] = X[v] = p;
        else update(u), update(v);
        return p;
    };
    auto flip = [&](int u, int c1, int c2) {
        int p = C[u][c1];
        swap(C[u][c1], C[u][c2]);
        if (p) G[u][p] = G[p][u] = c2;
        if (!C[u][c1]) X[u] = c1;
        if (!C[u][c2]) X[u] = c2;
        return p;
    };
    fill_n(X + 1, n, 1);
    FOR (t, 0, SZ(E) - 1) {
        int u = E[t].F, v0 = E[t].S, v = v0, c0 = X[u],
            c = c0, d;
        vector<pair<int, int>> L;
        fill_n(vst + 1, n, 0);
        while (!G[u][v0]) {
            L.emplace_back(v, d = X[v]);
            if (!C[v][c])
                for (int a = SZ(L) - 1; a >= 0; a--)
                    c = color(u, L[a].F, c);
            else if (!C[u][d])
                for (int a = SZ(L) - 1; a >= 0; a--)
                    color(u, L[a].F, L[a].S);
            else if (vst[d]) break;
            else vst[d] = 1, v = C[u][d];
        }
        if (!G[u][v0]) {
            for (; v; v = flip(v, c, d), swap(c, d));
            if (int a; C[u][c0]) {
                for (a = SZ(L) - 2; a >= 0 && L[a].S != c;

```

```

--a);
        for (; a >= 0; a--) color(u, L[a].F, L[a].S);
    }
    else --t;
}
} // namespace vizing

```

2.12 Minimum Clique Cover* [74e60d]

```

struct Clique_Cover { // 0-base, O(n2^n)
    int co[1 << N], n, E[N];
    int dp[1 << N];
    void init(int _n) {
        n = _n, fill_n(dp, 1 << n, 0);
        fill_n(E, n, 0), fill_n(co, 1 << n, 0);
    }
    void add_edge(int u, int v) {
        E[u] |= 1 << v, E[v] |= 1 << u;
    }
    int solve() {
        FOR (i, 0, n - 1) co[1 << i] = E[i] | (1 << i);
        co[0] = (1 << n) - 1;
        dp[0] = (n & 1) * 2 - 1;
        FOR (i, 1, (1 << n) - 1) {
            int t = i & -i;
            dp[i] = -dp[i ^ t];
            co[i] = co[i ^ t] & co[t];
        }
        FOR (i, 0, (1 << n) - 1) co[i] = (co[i] & i) == i;
        fwt(co, 1 << n, 1);
        FOR (ans, 1, n - 1) {
            int sum = 0; // probabilistic
            FOR (i, 0, (1 << n) - 1) sum += (dp[i] * co[i]);
            if (sum) return ans;
        }
        return n;
    }
};

```

2.13 NumberofMaximalClique* [785d70]

```

struct BronKerbosch { // 1-base
    int n, a[N], g[N][N];
    int cnt, all[N][N], some[N][N], none[N][N];
    void init(int _n) {
        n = _n;
        FOR (i, 1, n)
            FOR (j, 1, n) g[i][j] = 0;
    }
    void add_edge(int u, int v) {
        g[u][v] = g[v][u] = 1;
    }
    void dfs(int d, int an, int sn, int nn) {
        if (cnt > 1000) return; // pruning
        if (sn == 0 && nn == 0) ++cnt;
        int u = some[d][0];
        FOR (i, 0, sn - 1) {
            int v = some[d][i];
            if (g[u][v]) continue;
            int tsn = 0, tnn = 0;
            copy_n(all[d], an, all[d + 1]);
            all[d + 1][an] = v;
            FOR (j, 0, sn - 1)
                if (g[v][some[d][j]])
                    some[d + 1][tsn++] = some[d][j];
            FOR (j, 0, nn - 1)
                if (g[v][none[d][j]])
                    none[d + 1][tnn++] = none[d][j];
            dfs(d + 1, an + 1, tsn, tnn);
            some[d][i] = 0, none[d][nn++] = v;
        }
    }
    int solve() {
        iota(some[0], some[0] + n, 1);
        cnt = 0, dfs(0, 0, n, 0);
        return cnt;
    }
};

```

3 Data Structure

3.1 Discrete Trick

```

#define ALL(v) v.begin(), v.end()
vector<int> val;
// build
sort(ALL(val)), val.erase(unique(ALL(val)), val.end());

```

```
// index of x
upper_bound(ALL(val), x) - val.begin();
// max idx <= x
upper_bound(ALL(val), x) - val.begin();
// max idx < x
lower_bound(ALL(val), x) - val.begin();
```

3.2 Mergeable Heap [22fc92]

```
template <class T, class Info> // lazy add tag
struct min_heap { // mergeable
    using P = pair<T, Info>;
    priority_queue<P, vector<P>, greater<P>> pq;
    T lazy = 0;
    void push(P v) { pq.emplace(v.F - lazy, v.S); }
    P top() { return P(pq.top().F + lazy, pq.top().S); }
    void join(min_heap &rgt) {
        if ((int)pq.size() < (int)rgt.pq.size()) {
            swap(pq, rgt.pq);
            swap(lazy, rgt.lazy);
        }
        while (!rgt.pq.empty()) {
            push(rgt.top());
            rgt.pop();
        }
    }
    void pop() { pq.pop(); }
    bool empty() { return pq.empty(); }
    void add_lazy(T v) { lazy += v; }
};
```

3.3 BIT kth* [e39485]

```
int bit[N + 1]; // N = 2 ^ k
int query_kth(int k) {
    int res = 0;
    for (int i = N >> 1; i >= 1; i >>= 1)
        if (bit[res + i] < k)
            k -= bit[res + i];
    return res + 1;
}
```

3.4 Interval Container* [26ec16]

```
typedef pair<int, int> pii;
/* Add
    and remove intervals from a set of disjoint intervals.
    * Will merge the added interval with
    any overlapping intervals in the set when adding.
    * Intervals are [inclusive, exclusive). */
set<pii>
>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->F <= R) {
        R = max(R, it->S);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->S >= L) {
        L = min(L, it->F);
        R = max(R, it->S);
        is.erase(it);
    }
    return is.insert(before, pii(L, R));
}
void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->S;
    if (it->F == L) is.erase(it);
    else (int &)it->S = L;
    if (R != r2) is.emplace(R, r2);
}
```

3.5 Heavy light Decomposition* [37e0af]

```
struct Heavy_light_Decomposition { // 1-base
    int n, ulink[N], deep[N], mxson[N], w[N], pa[N];
    int t, pl[N], data[N], val[N]; // val: vertex data
    vector<int> G[N];
    void init(int _n) {
        n = _n;
        FOR (i, 1, n) G[i].clear(), mxson[i] = 0;
    }
    void add_edge(int a, int b) {
        G[a].pb(b), G[b].pb(a);
    }
    void dfs(int u, int f, int d) {
        w[u] = 1, pa[u] = f, deep[u] = d++;
```

```
        for (int &i : G[u])
            if (i != f) {
                dfs(i, u, d), w[u] += w[i];
                if (w[mxson[u]] < w[i]) mxson[u] = i;
            }
    }
    void cut(int u, int link) {
        data[pl[u] = ++t] = val[u], ulink[u] = link;
        if (!mxson[u]) return;
        cut(mxson[u], link);
        for (int i : G[u])
            if (i != pa[u] && i != mxson[u])
                cut(i, i);
    }
    void build() { dfs(1, 1, 1), cut(1, 1); }
    int query(int a, int b) {
        int ta = ulink[a], tb = ulink[b], res = 0;
        while (ta != tb) {
            if (deep[ta] > deep[tb]) swap(ta, tb), swap(a, b);
            // res += query(pl[tb], pl[b])
            tb = ulink[b = pa[tb]];
        }
        if (pl[a] > pl[b]) swap(a, b);
        // res += query(pl[a], pl[b])
        return res;
    }
};
```

3.6 Centroid Decomposition* [cbc02b]

```
typedef pair<ll, ll> pll;
struct Cent_Dec { // 1-base
    vector<pll> G[N];
    pll info[N]; // store info. of itself
    pll upinfo[N]; // store info. of climbing up
    int n, pa[N], layer[N], sz[N], done[N];
    ll dis[__lg(N) + 1][N];
    void init(int _n) {
        n = _n, layer[0] = -1;
        fill_n(pa + 1, n, 0), fill_n(done + 1, n, 0);
        FOR (i, 1, n) G[i].clear();
    }
    void add_edge(int a, int b, int w) {
        G[a].pb(pll(b, w)), G[b].pb(pll(a, w));
    }
    void get_cent(
        int u, int f, int &mx, int &c, int num) {
        int mxsz = 0;
        sz[u] = 1;
        for (pll e : G[u])
            if (!done[e.F] && e.F != f) {
                get_cent(e.F, u, mx, c, num);
                sz[u] += sz[e.F], mxsz = max(mxsz, sz[e.F]);
            }
        if (mx > max(mxsz, num - sz[u]))
            mx = max(mxsz, num - sz[u]), c = u;
    }
    void dfs(int u, int f, ll d, int org) {
        // if required, add self info or climbing info
        dis[layer[org]][u] = d;
        for (pll e : G[u])
            if (!done[e.F] && e.F != f)
                dfs(e.F, u, d + e.S, org);
    }
    int cut(int u, int f, int num) {
        int mx = 1e9, c = 0, lc;
        get_cent(u, f, mx, c, num);
        done[c] = 1, pa[c] = f, layer[c] = layer[f] + 1;
        for (pll e : G[c])
            if (!done[e.F]) {
                if (sz[e.F] > sz[c])
                    lc = cut(e.F, c, num - sz[c]);
                else lc = cut(e.F, c, sz[e.F]);
                upinfo[lc] = pll(), dfs(e.F, c, e.S, c);
            }
        return done[c] = 0, c;
    }
    void build() { cut(1, 0, n); }
    void modify(int u) {
        for (int a = u, ly = layer[a]; a;
            a = pa[a], --ly) {
            info[a].F += dis[ly][u], ++info[a].S;
            if (pa[a])
                upinfo[a].F += dis[ly - 1][u], ++upinfo[a].S;
        }
    }
    ll query(int u) {
        ll rt = 0;
        for (int a = u, ly = layer[a]; a;
```

```

    a = pa[a], --ly) {
    rt += info[a].F + info[a].S * dis[ly][u];
    if (pa[a])
        rt -=
            upinfo[a].F + upinfo[a].S * dis[ly - 1][u];
    }
    return rt;
}
};

```

3.7 Treap* [5ab1a1]

```

struct node {
    int data, sz;
    node *l, *r;
    node(int k) : data(k), sz(1), l(0), r(0) {}
    void up() {
        sz = 1;
        if (l) sz += l->sz;
        if (r) sz += r->sz;
    }
    void down() {}
};

int sz(node *a) { return a ? a->sz : 0; }
node *merge(node *a, node *b) {
    if (!a || !b) return a ? a : b;
    if (rand() % (sz(a) + sz(b)) < sz(a))
        return a->down(), a->r = merge(a->r, b), a->up(), a;
    return b->down(), b->l = merge(a, b->l), b->up(), b;
}

void split(node *o, node *&a, node *&b, int k) {
    if (!o) return a = b = 0, void();
    o->down();
    if (o->data <= k)
        a = o, split(o->r, a->r, b, k), a->up();
    else b = o, split(o->l, a, b->l, k), b->up();
}

void split2(node *o, node *&a, node *&b, int k) {
    if (sz(o) <= k) return a = o, b = 0, void();
    o->down();
    if (sz(o->l) + 1 <= k)
        a = o, split2(o->r, a->r, b, k - sz(o->l) - 1);
    else b = o, split2(o->l, a, b->l, k);
    o->up();
}

node *kth(node *o, int k) {
    if (k <= sz(o->l)) return kth(o->l, k);
    if (k == sz(o->l) + 1) return o;
    return kth(o->r, k - sz(o->l) - 1);
}

int Rank(node *o, int key) {
    if (!o) return 0;
    if (o->data < key)
        return sz(o->l) + 1 + Rank(o->r, key);
    else return Rank(o->l, key);
}

bool erase(node *&o, int k) {
    if (!o) return 0;
    if (o->data == k) {
        node *t = o;
        o->down(), o = merge(o->l, o->r);
        delete t;
        return 1;
    }
    node *&t = k < o->data ? o->l : o->r;
    return erase(t, k) ? o->up(), 1 : 0;
}

void insert(node *&o, int k) {
    node *a, *b;
    split(o, a, b, k),
        o = merge(a, merge(new node(k), b));
}

void interval(node *&o, int l, int r) {
    node *a, *b, *c;
    split2(o, a, b, l - 1), split2(b, b, c, r);
    // operate
    o = merge(a, merge(b, c));
}

```

3.8 LiChaoST* [4a4bee]

```

struct L {
    ll m, k, id;
    L() : id(-1) {}
    L(ll a, ll b, ll c) : m(a), k(b), id(c) {}
    ll at(ll x) { return m * x + k; }
};

class LiChao { // maintain max
private:

```

```

    int n; vector<L> nodes;
    void insert(int l, int r, int rt, L ln) {
        int m = (l + r) >> 1;
        if (nodes[rt].id == -1)
            return nodes[rt] = ln, void();
        bool atLeft = nodes[rt].at(l) < ln.at(l);
        if (nodes[rt].at(m) < ln.at(m))
            atLeft ^= 1, swap(nodes[rt], ln);
        if (r - l == 1) return;
        if (atLeft) insert(l, m, rt << 1, ln);
        else insert(m, r, rt << 1 | 1, ln);
    }

    ll query(int l, int r, int rt, ll x) {
        int m = (l + r) >> 1; ll ret = -INF;
        if (nodes[rt].id != -1) ret = nodes[rt].at(x);
        if (r - l == 1) return ret;
        if (x < m) return max(ret, query(l, m, rt << 1, x));
        return max(ret, query(m, r, rt << 1 | 1, x));
    }

public:
    LiChao(int n_) : n(n_), nodes(n * 4) {}
    void insert(L ln) { insert(0, n, 1, ln); }
    ll query(ll x) { return query(0, n, 1, x); }
};

```

3.9 Link cut tree* [047a48]

```

struct Splay { // path sum; swap + for ^ to get xor
    static Splay nil;
    Splay *ch[2], *f;
    int rev, size;
    ll val, sum;
    Splay (
        ll _val = 0) : rev(0), size(1), val(_val), sum(_val)
    { f = ch[0] = ch[1] = &nil; }
    bool isr()
    { return f->ch[0] != this && f->ch[1] != this; }
    int dir()
    { return f->ch[0] == this ? 0 : 1; }
    void setCh(Splay *c, int d) {
        ch[d] = c;
        if (c != &nil) c->f = this;
        pull();
    }
    void give_tag(int r) {
        if (r) swap(ch[0], ch[1]), rev ^= 1;
    }
    void push() {
        if (ch[0] != &nil) ch[0]->give_tag(rev);
        if (ch[1] != &nil) ch[1]->give_tag(rev);
        rev = 0;
    }
    void pull() {
        // take care of the nil!
        size = ch[0]->size + ch[1]->size + 1;
        sum = ch[0]->sum + ch[1]->sum + val;
        if (ch[0] != &nil) ch[0]->f = this;
        if (ch[1] != &nil) ch[1]->f = this;
    }
} Splay::nil;
Splay *nil = &Splay::nil;
void rotate(Splay *x) {
    Splay *p = x->f;
    int d = x->dir();
    if (!p->isr()) p->f->setCh(x, p->dir());
    else x->f = p->f;
    p->setCh(x->ch[!d], d);
    x->setCh(p, !d);
    p->pull(), x->pull();
}

void splay(Splay *x) {
    vector<Splay*> splayVec;
    for (Splay *q = x;; q = q->f) {
        splayVec.pb(q);
        if (q->isr()) break;
    }
    reverse(splayVec.begin(), splayVec.end());
    for (auto it : splayVec) it->push();
    while (!x->isr()) {
        if (x->f->isr()) rotate(x);
        else if (x->dir() == x->f->dir())
            rotate(x->f), rotate(x);
        else rotate(x), rotate(x);
    }
}

Splay* access(Splay *x) {
    Splay *q = nil;
    for (; x != nil; x = x->f)
        splay(x), x->setCh(q, 1), q = x;
}

```

```

    return q;
}
void root_path(Splay *x) { access(x), splay(x); }
void chroot(Splay *x){
    root_path(x), x->give_tag(1);
    x->push(), x->pull();
}
void split(Splay *x, Splay *y) {
    chroot(x), root_path(y);
}
void link(Splay *x, Splay *y) {
    root_path(x), chroot(y);
    x->setCh(y, 1);
}
void cut(Splay *x, Splay *y) {
    split(x, y);
    if (y->size != 5) return;
    y->push();
    y->ch[0] = y->ch[0]->f = nil;
}
Splay* get_root(Splay *x) {
    for (root_path(x); x->ch[0] != nil; x = x->ch[0])
        x->push();
    splay(x);
    return x;
}
bool conn(Splay *x, Splay *y) {
    return get_root(x) == get_root(y);
}
Splay* lca(Splay *x, Splay *y) {
    access(x), root_path(y);
    if (y->f == nil) return y;
    return y->f;
}
void change(Splay *x, ll val) {
    splay(x), x->val = val, x->pull();
}
ll query(Splay *x, Splay *y) {
    split(x, y);
    return y->sum;
}

```

3.10 KDTree [d79267]

```

namespace kdt {
int root, lc[maxn], rc[maxn], xl[maxn], xr[maxn],
    yl[maxn], yr[maxn];
point p[maxn];
int build(int l, int r, int dep = 0) {
    if (l == r) return -1;
    function<bool(const point &, const point &)> f =
        [dep](const point &a, const point &b) {
            if (dep & 1) return a.x < b.x;
            else return a.y < b.y;
        };
    int m = (l + r) >> 1;
    nth_element(p + l, p + m, p + r, f);
    xl[m] = xr[m] = p[m].x;
    yl[m] = yr[m] = p[m].y;
    lc[m] = build(l, m, dep + 1);
    if (~lc[m]) {
        xl[m] = min(xl[m], xl[lc[m]]);
        xr[m] = max(xr[m], xr[lc[m]]);
        yl[m] = min(yl[m], yl[lc[m]]);
        yr[m] = max(yr[m], yr[lc[m]]);
    }
    rc[m] = build(m + 1, r, dep + 1);
    if (~rc[m]) {
        xl[m] = min(xl[m], xl[rc[m]]);
        xr[m] = max(xr[m], xr[rc[m]]);
        yl[m] = min(yl[m], yl[rc[m]]);
        yr[m] = max(yr[m], yr[rc[m]]);
    }
    return m;
}
bool bound(const point &q, int o, long long d) {
    double ds = sqrt(d + 1.0);
    if (q.x < xl[o] - ds || q.x > xr[o] + ds ||
        q.y < yl[o] - ds || q.y > yr[o] + ds)
        return false;
    return true;
}
long long dist(const point &a, const point &b) {
    return (a.x - b.x) * 1ll * (a.x - b.x) +
        (a.y - b.y) * 1ll * (a.y - b.y);
}
void dfs(
    const point &q, long long &d, int o, int dep = 0) {
    if (!bound(q, o, d)) return;
}

```

```

long long cd = dist(p[o], q);
if (cd != 0) d = min(d, cd);
if ((dep & 1) && q.x < p[o].x ||
    !(dep & 1) && q.y < p[o].y) {
    if (~lc[o]) dfs(q, d, lc[o], dep + 1);
    if (~rc[o]) dfs(q, d, rc[o], dep + 1);
} else {
    if (~rc[o]) dfs(q, d, rc[o], dep + 1);
    if (~lc[o]) dfs(q, d, lc[o], dep + 1);
}
}
void init(const vector<point> &v) {
    FOR (i, 0, (int) v.size() - 1) p[i] = v[i];
    root = build(0, v.size());
}
long long nearest(const point &q) {
    long long res = 1e18;
    dfs(q, res, root);
    return res;
}
} // namespace kdt

```

4 Flow/Matching

4.1 Bipartite Matching* [a5746e]

```

struct Bipartite_Matching { // 0-base
int mp[N], mq[N], dis[N + 1], cur[N], l, r;
vector<int> G[N + 1];
bool dfs(int u) {
    for (int &i = cur[u]; i < (int)G[u].size(); i++) {
        int e = G[u][i];
        if (mq[e] ==
            l || (dis[mq[e]] == dis[u] + 1 && dfs(mq[e])))
            return mp[mq[e] = u] = e, 1;
    }
    return dis[u] = -1, 0;
}
bool bfs() {
    queue<int> q;
    fill_n(dis, l + 1, -1);
    FOR (i, 0, l - 1)
        if (!mp[i])
            q.push(i), dis[i] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int e : G[u])
            if (!dis[mq[e]])
                q.push(mq[e]), dis[mq[e]] = dis[u] + 1;
    }
    return dis[l] != -1;
}
int matching() {
    int res = 0;
    fill_n(mp, l, -1), fill_n(mq, r, l);
    while (bfs()) {
        fill_n(cur, l, 0);
        FOR (i, 0, l - 1) res += (!mp[i] && dfs(i));
    }
    return res; // (i, mp[i] != -1)
}
vector<int> cover() { // Konig, call after matching()
    vector<int> vl(l), vr(r), q, res; // L: i, R: l + e
    FOR (i, 0, l - 1) if (!mp[i]) vl[i] = 1, q.pb(i);
    for (int h = 0; h < (int)q.size(); h++)
        for (int e : G[q[h]]) if (!vr[e]) {
            vr[e] = 1;
            if (mq[e] != l && !vl[mq[e]])
                vl[mq[e]] = 1, q.pb(mq[e]);
        }
    FOR (i, 0, l - 1) if (!vl[i]) res.pb(i);
    FOR (e, 0, r - 1) if (vr[e]) res.pb(l + e);
    return res; // complement = max independent set
}
void add_edge(int s, int t) { G[s].pb(t); }
void init(int _l, int _r) {
    l = _l, r = _r;
    FOR (i, 0, l) G[i].clear();
}
};

```

4.2 Kuhn Munkres* [a7dd20]

```

struct KM { // 0-base, maximum matching
ll w[N][N], hl[N], hr[N], slk[N];
int fl[N], fr[N], pre[N], qu[N], ql, qr, n, m; // n <= m
bool vl[N], vr[N];
void init(int _n, int _m) {
}
}

```

```

    n = _n, m = _m;
    FOR (i, 0, n - 1) fill_n
        (w[i], m, 0); // perfect matching set to -INF
}
void add_edge(int a, int b, ll wei) {
    w[a][b] = max(w[a][b], wei);
}
bool Check(int y) {
    if (vr[y] = 1, ~fr[y])
        return vl[qu[qr++]] = fr[y] = 1;
    while (~y) swap(y, fl[fr[y] = pre[y]]);
    return 0;
}
void bfs(int s) {
    fill_n
        (slk, m, INF), fill_n(vl, n, 0), fill_n(vr, m, 0);
    ql = qr = 0, qu[qr++] = s, vl[s] = 1;
    for (ll d;;) {
        while (ql < qr)
            for (int y = 0, x = qu[ql++]; y < m; y++)
                if (!vr[y] &&
                    slk[y] >= (d = hl[x] + hr[y] - w[x][y])) {
                    if (pre[y] = x, d) slk[y] = d;
                    else if (!Check(y)) return;
                }
        d = INF;
        FOR (y, 0, m - 1)
            if (!vr[y] && d > slk[y]) d = slk[y];
        FOR (y, 0, m - 1) {
            if (vr[y]) hr[y] += d;
            else slk[y] -= d;
        }
        FOR (x, 0, n - 1) if (vl[x]) hl[x] -= d;
        FOR (y, 0, m - 1)
            if (!vr[y] && !slk[y] && !Check(y)) return;
    }
}
ll solve() {
    fill_n
        (fl, n, -1), fill_n(fr, m, -1), fill_n(hr, m, 0);
    FOR (
        i, 0, n - 1) hl[i] = *max_element(w[i], w[i] + m);
    FOR (i, 0, n - 1) bfs(i);
    ll res = 0;
    FOR (i, 0, n - 1) if (fl[i] != -1) res += w[i][fl[i]];
    return res;
}
};

```

4.3 MincostMaxflow* [cdf5a9]

```

#include <ext/pb_ds/priority_queue.hpp> // decrease-key
const ll INF = numeric_limits<ll>::max() / 4;
struct MinCostMaxFlow { // 0-base, need global N
    struct Edge {
        int from, to, rev;
        ll cap, cost, flow;
    } *par[N];
    vector<Edge> g[N];
    int n, vis[N];
    ll dis[N], pot[N];
    void init(int _n) {
        n = _n, fill_n(pot, n, 0);
        FOR (i, 0, n - 1) g[i].clear();
    }
    void add_edge(int a, int b, ll cap, ll cost) {
        if (a == b) return;
        g[a].pb(
            Edge{a, b, (int)g[b].size(), cap, cost, 0});
        g[b].pb(
            Edge{b, a, (int)g[a].size() - 1, 0, -cost, 0});
    }
    void path(int s) { // dijkstra on reduced costs
        fill_n(vis, n, 0), fill_n(dis, n, INF);
        dis[s] = 0;
        __gnu_pbds::priority_queue<pair<ll, int>> pq;
        vector<decltype(pq)::point_iterator> it(n);
        pq.push({0, s});
        while (!pq.empty()) {
            s = pq.top().S, pq.pop(), vis[s] = 1;
            ll d = dis[s] + pot[s];
            for (Edge &e : g[s])
                if (!vis[e.to]) {
                    ll v = d - pot[e.to] + e.cost;
                    if (e.cap - e.flow > 0 && v < dis[e.to]) {
                        dis[e.to] = v, par[e.to] = &e;
                        if (it[e.to] == pq.end())
                            it[e.to] = pq.push({-v, e.to});
                        else pq.modify(it[e.to], {-v, e.to});
                    }
                }
            }
        }
    }
};

```

```

    }
}
}
FOR (i, 0, n - 1)
    pot[i] = min(pot[i] + dis[i], INF);
}
pair<ll, ll> maxflow(int s, int t) {
    ll flow = 0, cost = 0;
    while (path(s), vis[t]) {
        ll f = INF;
        for (Edge *e = par[t]; e; e = par[e->from])
            f = min(f, e->cap - e->flow);
        flow += f;
        for (Edge *e = par[t]; e; e = par[e->from])
            e->flow += f, g[e->to][e->rev].flow -= f;
    }
    FOR (i, 0, n - 1)
        for (Edge &e : g[i]) cost += e.cost * e.flow;
    return {flow, cost / 2};
}
void setpi(int s) { // iff some cost is negative
    fill_n(pot, n, INF), pot[s] = 0;
    int it = n, ch = 1;
    ll v;
    while (ch-- && it--)
        FOR (i, 0, n - 1)
            if (pot[i] != INF)
                for (Edge &e : g[i])
                    if (e.cap)
                        if ((v = pot[i] + e.cost) < pot[e.to])
                            pot[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};

```

4.4 Maximum Simple Graph Matching* [9af74d]

```

struct Matching { // 0-base
    queue<int> q; int n;
    vector<int> fa, s, vis, pre, match;
    vector<vector<int>> G;
    int Find(int u)
    { return u == fa[u] ? u : fa[u] = Find(fa[u]); }
    int LCA(int x, int y) {
        static int tk = 0; tk++; x = Find(x); y = Find(y);
        for (; swap(x, y)) if (x != n) {
            if (vis[x] == tk) return x;
            vis[x] = tk;
            x = Find(pre[match[x]]);
        }
    }
    void Blossom(int x, int y, int l) {
        for (; Find(x) != l; x = pre[y]) {
            pre[x] = y, y = match[x];
            if (s[y] == 1) q.push(y), s[y] = 0;
            for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
        }
    }
    bool Bfs(int r) {
        iota(fa.begin(), fa.end(), 0);
        fill(s.begin(), s.end(), -1);
        q = queue<int>(); q.push(r); s[r] = 0;
        for (; !q.empty(); q.pop()) {
            for (int x = q.front(); int u : G[x])
                if (s[u] == -1) {
                    if (pre[u] = x, s[u] = 1, match[u] == n) {
                        for (int a = u, b = x, last;
                            b != n; a = last, b = pre[a])
                            last = match[b], match[b] = a, match[a] = b;
                        return true;
                    }
                    q.push(match[u]); s[match[u]] = 0;
                } else if (!s[u] && Find(u) != Find(x)) {
                    int l = LCA(u, x);
                    Blossom(x, u, l); Blossom(u, x, l);
                }
            }
        }
        return false;
    }
    Matching(int _n) : n(_n), fa(n + 1), s(n + 1),
        vis(n + 1), pre(n + 1, n), match(n + 1, n), G(n) {}
    void add_edge(int u, int v)
    { G[u].pb(v), G[v].pb(u); }
    int solve() {
        int ans = 0;
        FOR (x, 0, n - 1)
            if (match[x] == n) ans += Bfs(x);
        return ans;
    }
    // match[x] == n means not matched
};

```

};

4.5 Maximum Weight Matching* [354226]

```
#define ALL(v) v.begin(), v.end()
struct WeightGraph { // 1-based
    struct edge { int u, v, w; }; int n, nx;
    vector<int> lab; vector<vector<edge>> g;
    vector<int> slk, match, st, pa, sl, vis;
    vector<vector<int>> flo, flo_from; queue<int> q;
    WeightGraph(int n_) : n(n_), nx(n * 2), lab(n + 1),
        g(nx + 1, vector<edge>(nx + 1)), slk(nx + 1),
        flo(nx + 1, flo_from(nx + 1, vector(n + 1, 0))) {
        match = st = pa = sl = vis = slk;
        FOR (u, 1, n)
            FOR (v, 1, n) g[u][v] = {u, v, 0};
    }
    int E(edge e)
    { return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
    void update_slk(int u, int x, int &s)
    { if (!s || E(g[u][x]) < E(g[s][x])) s = u; }
    void set_slk(int x) {
        slk[x] = 0;
        FOR (u, 1, n)
            if (g[u][x].w > 0 && st[u] != x &&
                sl[st[u]] == 0)
                update_slk(u, x, slk[x]);
    }
    void q_push(int x) {
        if (x <= n) q.push(x);
        else for (int y : flo[x]) q_push(y);
    }
    void set_st(int x, int b) {
        st[x] = b;
        if (x > n) for (int y : flo[x]) set_st(y, b);
    }
    vector<int> split_flo(auto &f, int xr) {
        auto it = find(ALL(f), xr);
        if (auto pr = it - f.begin(); pr % 2 == 1)
            reverse(1 + ALL(f)), it = f.end() - pr;
        auto res = vector(f.begin(), it);
        return f.erase(f.begin(), it), res;
    }
    void set_match(int u, int v) {
        match[u] = g[u][v].v;
        if (u <= n) return;
        int xr = flo_from[u][g[u][v].u];
        auto &f = flo[u], z = split_flo(f, xr);
        FOR (i, 0, (int)z.size() - 1)
            set_match(z[i], z[i ^ 1]);
        set_match(xr, v); f.insert(f.end(), ALL(z));
    }
    void augment(int u, int v) {
        for (;;) {
            int xnv = st[match[u]]; set_match(u, v);
            if (!xnv) return;
            set_match(v = xnv, u = st[pa[xnv]]);
        }
    }
    int lca(int u, int v) {
        static int t = 0; ++t;
        for (++t; u || v; swap(u, v)) if (u) {
            if (vis[u] == t) return u;
            vis[u] = t, u = st[match[u]];
            if (u) u = st[pa[u]];
        }
        return 0;
    }
    void add_blossom(int u, int o, int v) {
        int b = find(n + 1 + ALL(st), 0) - begin(st);
        lab[b] = 0, sl[b] = 0, match[b] = match[o];
        vector<int> f = {o};
        for (int t : {u, v}) {
            reverse(1 + ALL(f));
            for (int x = t, y; x != o; x = st[pa[y]])
                f.pb(x), f.pb(y = st[match[x]]), q_push(y);
        }
        flo[b] = f; set_st(b, b);
        FOR (x, 1, nx) g[b][x].w = g[x][b].w = 0;
        fill(ALL(flo_from[b]), 0);
        for (int xs : flo[b]) {
            FOR (x, 1, nx)
                if (g[b][x].w == 0 || E(g[xs][x]) < E(g[b][x]))
                    g[b][x] = g[xs][x], g[x][b] = g[x][xs];
            FOR (x, 1, n)
                if (flo_from[xs][x]) flo_from[b][x] = xs;
        }
        set_slk(b);
    }
};
```

```
void expand_blossom(int b) {
    for (int x : flo[b]) set_st(x, x);
    int xr = flo_from[b][g[b][pa[b]].u], xs = -1;
    for (int x : split_flo(flo[b], xr)) {
        if (xs == -1) { xs = x; continue; }
        pa[xs] = g[x][xs].u, sl[xs] = 1, sl[x] = 0;
        slk[xs] = 0, set_slk(x), q_push(x), xs = -1;
    }
    for (int x : flo[b])
        if (x == xr) sl[x] = 1, pa[x] = pa[b];
        else sl[x] = -1, set_slk(x);
    st[b] = 0;
}
bool on_found_edge(const edge &e) {
    if (int u = st[e.u], v = st[e.v]; sl[v] == -1) {
        int nu = st[match[v]];
        pa[v] = e.u;
        sl[v] = 1;
        slk[v] = slk[nu] = sl[nu] = 0;
        q_push(nu);
    } else if (sl[v] == 0) {
        if (int o = lca(u, v)) add_blossom(u, o, v);
        else return augment(u, v), augment(v, u), true;
    }
    return false;
}
bool matching() {
    fill(ALL(sl), -1), fill(ALL(slk), 0);
    q = queue<int>();
    FOR (x, 1, nx)
        if (st[x] == x && !match[x])
            pa[x] = sl[x] = 0, q_push(x);
    if (q.empty()) return false;
    for (;;) {
        while ((int)q.size()) {
            int u = q.front(); q.pop();
            if (sl[st[u]] == 1) continue;
            FOR (v, 1, n)
                if (g[u][v].w > 0 && st[u] != st[v]) {
                    if (E(g[u][v]) != 0)
                        update_slk(u, st[v], slk[st[v]]);
                    else if (on_found_edge(g[u][v])) return true;
                }
        }
        int d = INF;
        FOR (b, n + 1, nx)
            if (st[b] == b && sl[b] == 1)
                d = min(d, lab[b] / 2);
        FOR (x, 1, nx)
            if (int s = slk[x];
                st[x] == x && s && sl[x] <= 0)
                d = min(d, E(g[s][x]) / (sl[x] + 2));
        FOR (u, 1, n)
            if (sl[st[u]] == 1) lab[u] += d;
            else if (sl[st[u]] == 0) {
                if (lab[u] <= d) return false;
                lab[u] -= d;
            }
        FOR (b, n + 1, nx)
            if (st[b] == b && sl[b] >= 0)
                lab[b] += d * (2 - 4 * sl[b]);
        FOR (x, 1, nx)
            if (int s = slk[x]; st[x] == x &&
                s && st[s] != x && E(g[s][x]) == 0)
                if (on_found_edge(g[s][x])) return true;
        FOR (b, n + 1, nx)
            if (st[b] == b && sl[b] == 1 && lab[b] == 0)
                expand_blossom(b);
    }
    return false;
}
pair<ll, int> solve() {
    fill(ALL(match), 0);
    FOR (u, 0, n) st[u] = u, flo[u].clear();
    int w_max = 0;
    FOR (u, 1, n)
        FOR (v, 1, n) {
            flo_from[u][v] = (u == v ? u : 0);
            w_max = max(w_max, g[u][v].w);
        }
    fill(ALL(lab), w_max);
    int n_matches = 0; ll tot_weight = 0;
    while (matching()) ++n_matches;
    FOR (u, 1, n)
        if (match[u] && match[u] < u)
            tot_weight += g[u][match[u]].w;
    return make_pair(tot_weight, n_matches);
}
```

```
void add_edge(int u, int v, int w)
{ g[u][v].w = g[v][u].w = w; }
};
```

4.6 SW-mincut [524fe7]

```
pair<int, vector<int>> globalMinCut(vector<vector<int>> mat) {
    pair<int, vector<int>> best = {INT_MAX, {}};
    int n = mat.size();
    vector<vector<int>> co(n);
    FOR (i, 0, n - 1) co[i] = {i};
    FOR (ph, 1, n - 1) {
        vector<int> w = mat[0];
        int s = 0, t = 0;
        FOR (it, 0, n - ph - 1) { //  $O(V^2) \rightarrow O(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t,
            t = max_element(w.begin(), w.end()) - w.begin();
            FOR (i, 0, n - 1) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), co[t].begin(), co[t].end());
        FOR (i, 0, n - 1) mat[s][i] += mat[t][i];
        FOR (i, 0, n - 1) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
}
```

4.7 BoundedFlow*(Dinic*) [e2d282]

```
#define SZ(a) ((int)a.size())
struct BoundedFlow { // 0-base
    struct edge {
        int to, cap, flow, rev;
    };
    vector<edge> G[N];
    int n, s, t, dis[N], cur[N], cnt[N];
    void init(int _n) {
        n = _n;
        FOR (i, 0, n + 1) G[i].clear(), cnt[i] = 0;
    }
    void add_edge(int u, int v, int lcap, int rcap) {
        cnt[u] -= lcap, cnt[v] += lcap;
        G[u].pb(edge{v, rcap, lcap, SZ(G[v])});
        G[v].pb(edge{u, 0, 0, SZ(G[u]) - 1});
    }
    void add_edge(int u, int v, int cap) {
        G[u].pb(edge{v, cap, 0, SZ(G[v])});
        G[v].pb(edge{u, 0, 0, SZ(G[u]) - 1});
    }
    int dfs(int u, int cap) {
        if (u == t || !cap) return cap;
        for (int &i = cur[u]; i < SZ(G[u]); i++) {
            edge &e = G[u][i];
            if (dis[e.to] == dis[u] + 1 && e.cap != e.flow) {
                int df = dfs(e.to, min(e.cap - e.flow, cap));
                if (df) {
                    e.flow += df, G[e.to][e.rev].flow -= df;
                    return df;
                }
            }
        }
        dis[u] = -1;
        return 0;
    }
    bool bfs() {
        fill_n(dis, n + 3, -1);
        queue<int> q;
        q.push(s), dis[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (edge &e : G[u])
                if (!dis[e.to] && e.flow != e.cap)
                    q.push(e.to), dis[e.to] = dis[u] + 1;
        }
        return dis[t] != -1;
    }
    int maxflow(int _s, int _t) {
        s = _s, t = _t;
        int flow = 0, df;
        while (bfs()) {
            fill_n(cur, n + 3, 0);
            while ((df = dfs(s, INF))) flow += df;
        }
        return flow;
    }
};
```

```
bool solve() {
    int sum = 0;
    FOR (i, 0, n - 1)
        if (cnt[i] > 0)
            add_edge(n + 1, i, cnt[i]), sum += cnt[i];
        else if (cnt[i] < 0) add_edge(i, n + 2, -cnt[i]);
    if (sum != maxflow(n + 1, n + 2)) sum = -1;
    FOR (i, 0, n - 1)
        if (cnt[i] > 0)
            G[n + 1].pop_back(), G[i].pop_back();
        else if (cnt[i] < 0)
            G[i].pop_back(), G[n + 2].pop_back();
    return sum != -1;
}
int solve(int _s, int _t) {
    add_edge(_t, _s, INF);
    if (!solve()) return -1; // invalid flow
    int x = G[_t].back().flow;
    return G[_t].pop_back(), G[_s].pop_back(), x;
}
};
```

4.8 Gomory Hu tree* [19c609]

```
MaxFlow Dinic;
int g[MXN];
void GomoryHu(int n) { // 0-base
    fill_n(g, n, 0);
    FOR (i, 1, n - 1) {
        Dinic.reset();
        add_edge(i, g[i], Dinic.maxflow(i, g[i]));
        FOR (j, i + 1, n)
            if (g[j] == g[i] && ~Dinic.dis[j])
                g[j] = i;
    }
}
```

4.9 Minimum Cost Circulation* [b3f178]

```
struct MinCostCirculation { // 0-base
    struct Edge {
        ll from, to, cap, fcap, flow, cost, rev;
    } *past[N];
    vector<Edge> G[N];
    ll dis[N], inq[N], n;
    void BellmanFord(int s) {
        fill_n(dis, n, INF), fill_n(inq, n, 0);
        queue<int> q;
        auto relax = [&](int u, ll d, Edge *e) {
            if (dis[u] > d) {
                dis[u] = d, past[u] = e;
                if (!inq[u]) inq[u] = 1, q.push(u);
            }
        };
        relax(s, 0, 0);
        while (!q.empty()) {
            int u = q.front();
            q.pop(), inq[u] = 0;
            for (auto &e : G[u])
                if (e.cap > e.flow)
                    relax(e.to, dis[u] + e.cost, &e);
        }
    }
    void try_edge(Edge &cur) {
        if (cur.cap > cur.flow) return ++cur.cap, void();
        BellmanFord(cur.to);
        if (dis[cur.from] + cur.cost < 0) {
            ++cur.flow, --G[cur.to][cur.rev].flow;
            for (int i = cur.from; past[i]; i = past[i]->from) {
                auto &e = *past[i];
                ++e.flow, --G[e.to][e.rev].flow;
            }
        }
        ++cur.cap;
    }
    void solve(int mxlg) {
        for (int b = mxlg; b >= 0; b--) {
            FOR (i, 0, n - 1)
                for (auto &e : G[i])
                    e.cap *= 2, e.flow *= 2;
            FOR (i, 0, n - 1)
                for (auto &e : G[i])
                    if (e.fcap >> b & 1)
                        try_edge(e);
        }
    }
    void init(int _n) { n = _n;
        FOR (i, 0, n - 1) G[i].clear();
    }
};
```

```

}
void add_edge(ll a, ll b, ll cap, ll cost) {
    G[a].pb(Edge{a, b, 0, cap, 0, cost,
        (int)G[b].size() + (a == b)});
    G[b].pb(Edge{
        b, a, 0, 0, -cost, (int)G[a].size() - 1});
}
} mcmf; // O(VE * ElogC)

```

4.10 Flow Models

- Maximum/Minimum flow with lower bound / Circulation problem
 - Construct super source S and sink T .
 - For each edge (x, y, l, u) , connect $x \rightarrow y$ with capacity $u - l$.
 - For each vertex v , denote by $in(v)$ the difference between the sum of incoming lower bounds and the sum of outgoing lower bounds.
 - If $in(v) > 0$, connect $S \rightarrow v$ with capacity $in(v)$, otherwise, connect $v \rightarrow T$ with capacity $-in(v)$.
 - To maximize, connect $t \rightarrow s$ with capacity ∞ (skip this in circulation problem), and let f be the maximum flow from S to T . If $f \neq \sum_{v \in V, in(v) > 0} in(v)$, there's no solution. Otherwise, the maximum flow from s to t is the answer.
 - To minimize, let f be the maximum flow from S to T . Connect $t \rightarrow s$ with capacity ∞ and let the flow from S to T be f' . If $f + f' \neq \sum_{v \in V, in(v) > 0} in(v)$, there's no solution. Otherwise, f' is the answer.
 - The solution of each edge e is $l_e + f_e$, where f_e corresponds to the flow of edge e on the graph.
- Construct minimum vertex cover from maximum matching M on bipartite graph (X, Y)
 - Redirect every edge: $y \rightarrow x$ if $(x, y) \in M$, $x \rightarrow y$ otherwise.
 - DFS from unmatched vertices in X .
 - $x \in X$ is chosen iff x is unvisited.
 - $y \in Y$ is chosen iff y is visited.
- Minimum cost cyclic flow
 - Construct super source S and sink T
 - For each edge (x, y, c) , connect $x \rightarrow y$ with $(cost, cap) = (c, 1)$ if $c > 0$, otherwise connect $y \rightarrow x$ with $(cost, cap) = (-c, 1)$
 - For each edge with $c < 0$, sum these cost as K , then increase $d(y)$ by 1, decrease $d(x)$ by 1
 - For each vertex v with $d(v) > 0$, connect $S \rightarrow v$ with $(cost, cap) = (0, d(v))$
 - For each vertex v with $d(v) < 0$, connect $v \rightarrow T$ with $(cost, cap) = (0, -d(v))$
 - Flow from S to T , the answer is the cost of the flow $C + K$
- Maximum density induced subgraph
 - Binary search on answer, suppose we're checking answer T
 - Construct a max flow model, let K be the sum of all weights
 - Connect source $s \rightarrow v, v \in G$ with capacity K
 - For each edge (u, v, w) in G , connect $u \rightarrow v$ and $v \rightarrow u$ with capacity w
 - For $v \in G$, connect it with sink $v \rightarrow t$ with capacity $K + 2T - (\sum_{e \in E(v)} w(e)) - 2w(v)$
 - T is a valid answer if the maximum flow $f < K|V|$
- Minimum weight edge cover
 - For each $v \in V$ create a copy v' , and connect $u' \rightarrow v'$ with weight $w(u, v)$.
 - Connect $v \rightarrow v'$ with weight $2\mu(v)$, where $\mu(v)$ is the cost of the cheapest edge incident to v .
 - Find the minimum weight perfect matching on G' .
- Project selection problem
 - If $p_v > 0$, create edge (s, v) with capacity p_v ; otherwise, create edge (v, t) with capacity $-p_v$.
 - Create edge (u, v) with capacity w with w being the cost of choosing u without choosing v .
 - The mincut is equivalent to the maximum profit of a subset of projects.
- Dual of minimum cost maximum flow
 - Capacity c_{uv} , Flow f_{uv} , Cost w_{uv} , Required Flow difference for vertex b_u .
 - If all w_{uv} are integers, then optimal solution can happen when all p_u are integers.

$$\begin{aligned}
 \min \sum_{uv} w_{uv} f_{uv} \\
 -f_{uv} \geq -c_{uv} \Leftrightarrow \min \sum_u b_u p_u + \sum_{uv} c_{uv} \max(0, p_v - p_u - w_{uv}) \\
 \sum_v f_{vu} - \sum_v f_{uv} = -b_u \quad p_u \geq 0
 \end{aligned}$$

5 String

5.1 KMP [c665d9]

```

int fail[MAXN];
vector<int> match(string A, string B) {
    vector<int> ans;
    fail[0] = -1, fail[1] = 0;
    for (int i = 1, j = 0; i < (int)B.size();
        fail[++i] = ++j) {
        if (B[i] == B[j]) fail[i] = fail[j]; // optimize
        while (j != -1 && B[i] != B[j]) j = fail[j];
    }
    for (int i = 0, j = 0; i < (int)A.size(); i++) {
        while (j != -1 && A[i] != B[j]) j = fail[j];
        if (++j == (int)B.size())
            ans.pb(i + 1 - j), j = fail[j];
    }
}

```

```

}
return ans;
}

```

5.2 Z-value* [3dd732]

```

int z[SIZE];
void make_z(const string &s) {
    int l = 0, r = 0, n = s.size();
    FOR (i, 1, n - 1) {
        for (z[i] = max(0, min(r - i + 1, z[i - l]));
            i + z[i] < n && s[i + z[i]] == s[z[i]]; ++z[i]);
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
}

```

5.3 Manacher* [29eb4f]

```

int z[SIZE]; // 0-base
/* center i: radius z[i * 2 + 1] / 2
   center i, i + 1: radius z[i * 2 + 2] / 2
   both aba, abba have radius 2 */
void Manacher(string tmp) {
    string s = "%";
    int l = 0, r = 0, n;
    for (char c : tmp) s.push_back(c), s.push_back('%');
    n = s.size();
    FOR (i, 0, n - 1) {
        z[i] = r > i ? min(z[2 * l - i], r - i) : 1;
        while (i - z[i] >= 0 && i + z[i] < s.size()
            && s[i + z[i]] == s[i - z[i]]) ++z[i];
        if (z[i] + i > r) r = z[i] + i, l = i;
    }
}

```

5.4 Suffix Array* [df6e92]

```

vector<int> suffix_array(string s) {
    int n = s.size();
    if (n == 1) return {0};
    s += '\0';
    vector<int> sa(n + 1), rnk(n + 1), cnt(n + 1);
    array<int, 256> alp{};
    FOR (i, 0, n) alp[s[i]]++;
    FOR (i, 1, 255) alp[i] += alp[i - 1];
    FOR (i, 0, n) sa[--alp[s[i]]] = i;
    for (int i = 1, cur = 0; i <= n; i++) {
        if (s[sa[i]] != s[sa[i - 1]]) cur++;
        rnk[sa[i]] = cur;
    }
    for (int len = 1; len <= n; len <= 1) {
        vector<int> lp(n + 1), nrnk(n + 1);
        fill(cnt.begin(), cnt.end(), 0);
        FOR (i, 0, n) {
            lp[i] = sa[i] - len;
            if (lp[i] < 0) lp[i] += n + 1;
            cnt[rnk[i]]++;
        }
        FOR (i, 1, n) cnt[i] += cnt[i - 1];
        for (int i = n; i >= 0; i--)
            sa[--cnt[rnk[lp[i]]]] = lp[i];
        nrnk[sa[0]] = 0;
        for (int i = 1, cur = 0; i <= n; i++) {
            int np1 = sa[i] + len, np2 = sa[i - 1] + len;
            if (np1 >= n + 1) np1 -= n + 1;
            if (np2 >= n + 1) np2 -= n + 1;
            if (rnk[sa[i]] != rnk[sa[i - 1]]) ||
                rnk[np1] != rnk[np2])
                cur++;
            nrnk[sa[i]] = cur;
        }
        rnk = nrnk;
    }
    sa.erase(sa.begin());
    return sa;
}
vector<int> lcp_array(string s, vector<int> sa = {}) {
    int n = s.size(), len = 0;
    if (sa.size() == 0) sa = suffix_array(s);
    vector<int> rnk(n), lcp(n - 1);
    FOR (i, 0, n - 1) rnk[sa[i]] = i;
    FOR (i, 0, n - 1) {
        if (rnk[i] == n - 1) {
            len = 0;
            continue;
        }
        int j = sa[rnk[i] + 1];
        while (i + len < n && j + len < n &&
            s[i + len] == s[j + len])

```

```

    len++;
    lcp[rnk[i]] = len;
    len = max(0, len - 1);
}
return lcp;
}

```

5.5 SAIS C++20* [6b45b6]

```

auto sais(const auto &s) {
    const int n = (int)s.size(), z = ranges::max(s) + 1;
    if (n == 1) return vector{0};
    vector<int> c(z); for (int x : s) ++c[x];
    partial_sum(c.begin(), c.end(), begin(c));
    vector<int> sa(n); auto I = views::iota(0, n);
    vector<bool> t(n, true);
    for (int i = n - 2; i >= 0; i--)
        t[i] = (s[i] == s[i + 1] ? t[i + 1] : s[i] < s[i + 1]);
    auto is_lms = views::filter([&t](int x) {
        return x && t[x] && !t[x - 1];
    });
    auto induce = [&] {
        for (auto x = c; int y : sa)
            if (y-- && (!t[y])) sa[x[s[y] - 1]++] = y;
        for (auto x = c; int y : sa | views::reverse)
            if (y-- && (t[y])) sa[--x[s[y]]] = y;
    };
    vector<int> lms, q(n); lms.reserve(n);
    for (auto x = c; int i : I | is_lms)
        q[i] = (int)lms.size(), lms.pb(sa[--x[s[i]]] = i);
    induce();
    vector<int> ns((int)lms.size());
    for (int j = -1, nz = 0; int i : sa | is_lms) {
        if (j >= 0) {
            int len = min({n - i, n - j, lms[q[i] + 1] - i});
            ns[q[i]] = nz += lexicographical_compare(
                begin(s) + j, begin(s) + j + len,
                begin(s) + i, begin(s) + i + len);
        }
        j = i;
    }
    fill(sa.begin(), sa.end(), 0);
    auto nsa = sais(ns);
    for (auto x = c; int y : nsa | views::reverse)
        y = lms[y], sa[--x[s[y]]] = y;
    return induce(), sa;
}
// sa[i]: sa[i]-th
// suffix is the i-th lexicographically smallest suffix.
// hi[i]: LCP of suffix sa[i] and suffix sa[i - 1].
struct Suffix { // 0-based
    int n; vector<int> sa, hi, ra;
    Suffix(const auto &s, int _n) : n(_n), hi(n), ra(n) {
        vector<int> s(n + 1); // s[n] = 0;
        copy_n(
            begin(s), n, begin(s)); // _s shouldn't contain 0
        sa = sais(s); sa.erase(sa.begin());
        for (int i = 0; i < n; i++) ra[sa[i]] = i;
        for (int i = 0, h = 0; i < n; i++) {
            if (!ra[i]) { h = 0; continue; }
            for (int j = sa[ra[i] - 1];
                max(i, j) + h < n && s[i + h] == s[j + h];) ++h;
            hi[ra[i]] = h ? h - 1 : 0;
        }
    }
};

```

5.6 Aho-Corasick Automatan* [dbd87e]

```

struct AC_Automatan {
    int nx[len][sigma], fl[len], cnt[len], ord[len], top;
    int rnx[len][sigma]; // node actually be reached
    int newnode() {
        fill_n(nx[top], sigma, -1);
        return top++;
    }
    void init() { top = 1, newnode(); }
    int input(string &s) {
        int X = 1;
        for (char c : s) {
            if (!nx[X][c - 'A']) nx[X][c - 'A'] = newnode();
            X = nx[X][c - 'A'];
        }
        return X; // return the end node of string
    }
    void make_fl() {
        queue<int> q;
        q.push(1), fl[1] = 0;
    }
}

```

```

for (int t = 0; !q.empty(); ) {
    int R = q.front();
    q.pop(), ord[t++] = R;
    FOR (i, 0, sigma - 1)
        if (~nx[R][i]) {
            int X = rnx[R][i] = nx[R][i], Z = fl[R];
            for (; Z && !~nx[Z][i]; ) Z = fl[Z];
            fl[X] = Z ? nx[Z][i] : 1, q.push(X);
        }
        else rnx[R][i] = R > 1 ? rnx[fl[R]][i] : 1;
    }
}
void solve() {
    for (int i = top - 2; i > 0; i--) // if you want
        to count all string occurrences in query string t
        cnt[fl[ord[i]]]
            += cnt[ord[i]]; // iterate in forward order and
        // propagate counts from ord[i] to fl[ord[i]]
}
} ac;

```

5.7 Smallest Rotation* [5323de]

```

string mcp(string s) {
    int n = (int)s.size(), i = 0, j = 1;
    s += s;
    while (i < n && j < n) {
        int k = 0;
        while (k < n && s[i + k] == s[j + k]) ++k;
        if (s[i + k] <= s[j + k]) j += k + 1;
        else i += k + 1;
        if (i == j) ++j;
    }
    int ans = i < n ? i : j;
    return s.substr(ans, n);
}

```

5.8 De Bruijn sequence* [18d8a1]

```

constexpr int MAXC = 10, MAXN = 1e5 + 10;
struct DBSeq {
    int C, N, K, L, buf[MAXC * MAXN]; // K <= C^N
    void dfs(int *out, int t, int p, int &ptr) {
        if (ptr >= L) return;
        if (t > N) {
            if (N % p) return;
            for (int i = 1; i <= p && ptr < L; i++)
                out[ptr++] = buf[i];
        } else {
            buf[t] = buf[t - p], dfs(out, t + 1, p, ptr);
            FOR (j, buf[t - p] + 1, C - 1)
                buf[t] = j, dfs(out, t + 1, t, ptr);
        }
    }
    void solve(int _c, int _n, int _k, int *out) {
        int p = 0;
        C = _c, N = _n, K = _k, L = N + K - 1;
        dfs(out, 1, 1, p);
        if (p < L) fill(out + p, out + L, 0);
    }
} dbs;

```

5.9 Extended SAM* [d31fc3]

```

struct exSAM {
    int len[N * 2], link[N * 2]; // maxlen, suflink
    int next[N * 2][CNUM], tot; // [0, tot), root = 0
    int lenSorted[N * 2]; // topo. order
    int cnt[N * 2]; // occurence
    int newnode() {
        fill_n(next[tot], CNUM, 0);
        len[tot] = cnt[tot] = link[tot] = 0;
        return tot++;
    }
    void init() { tot = 0, newnode(), link[0] = -1; }
    int insertSAM(int last, int c) {
        int cur = next[last][c];
        len[cur] = len[last] + 1;
        int p = link[last];
        while (p != -1 && !next[p][c])
            next[p][c] = cur, p = link[p];
        if (p == -1) return link[cur] = 0, cur;
        int q = next[p][c];
        if (len[p] + 1 == len[q]) return link[cur] = q, cur;
        int clone = newnode();
        FOR (i, 0, CNUM - 1)
            next[clone][i] = len[next[q][i]] ? next[q][i] : 0;
        len[clone] = len[p] + 1;
        while (p != -1 && next[p][c] == q)

```

```

    next[p][c] = clone, p = link[p];
    link[link[cur] = clone] = link[q];
    link[q] = clone;
    return cur;
}

void insert(const string &s) {
    int cur = 0;
    for (auto ch : s) {
        int &nxt = next[cur][int(ch - 'a')];
        if (!nxt) nxt = newnode();
        cnt[cur = nxt] += 1;
    }
}

void build() {
    queue<int> q;
    q.push(0);
    while (!q.empty()) {
        int cur = q.front();
        q.pop();
        FOR (i, 0, CNUM - 1)
            if (next[cur][i])
                q.push(insertSAM(cur, i));
    }
    vector<int> lc(tot);
    FOR (i, 1, tot - 1) ++lc[len[i]];
    partial_sum(lc.begin(), lc.end(), lc.begin());
    FOR (i, 1, tot - 1) lenSorted[--lc[len[i]]] = i;
}

void solve() {
    for (int i = tot - 2; i >= 0; i--)
        cnt[link[lenSorted[i]]] += cnt[lenSorted[i]];
}
};

```

5.10 PalTree* [189708]

```

struct palindromic_tree {
    struct node {
        int next[26], fail, len;
        int cnt, num; // cnt: appear times, num: number of
                       // pal. suf.
        node(int l = 0) : fail(0), len(l), cnt(0), num(0) {
            FOR (i, 0, 25) next[i] = 0;
        }
    };
    vector<node> St;
    vector<char> s;
    int last, n;
    palindromic_tree() : St(2), last(1), n(0) {
        St[0].fail = 1, St[1].len = -1, s.pb(-1);
    }
    inline void clear() {
        St.clear(), s.clear(), last = 1, n = 0;
        St.pb(0), St.pb(-1);
        St[0].fail = 1, s.pb(-1);
    }
    inline int get_fail(int x) {
        while (s[n - St[x].len - 1] != s[n])
            x = St[x].fail;
        return x;
    }
    inline void add(int c) {
        s.pb(c -= 'a'), ++n;
        int cur = get_fail(last);
        if (!St[cur].next[c]) {
            int now = (int)St.size();
            St.pb(St[cur].len + 2);
            St[now].fail =
                St[get_fail(St[cur].fail)].next[c];
            St[cur].next[c] = now;
            St[now].num = St[St[now].fail].num + 1;
        }
        last = St[cur].next[c], ++St[last].cnt;
    }
    inline void count() { // counting cnt
        auto i = St.rbegin();
        for (; i != St.rend(); ++i) {
            St[i->fail].cnt += i->cnt;
        }
    }
    inline int size() { // The number of diff. pal.
        return (int)St.size() - 2;
    }
};

```

5.11 Main Lorentz [62bca1]

```

vector<pair<int, int>> rep[kN]; // 0-base [l, r]
void main_lorentz(const string &s, int sft = 0) {

```

```

    const int n = s.size();
    if (n == 1) return;
    const int nu = n / 2, nv = n - nu;
    const string u = s.substr(0, nu), v = s.substr(nu),
        ru(u.rbegin(), u.rend()), rv(v.rbegin(), v.rend());
    main_lorentz(u, sft), main_lorentz(v, sft + nu);
    const auto z1 = Zalgo(ru), z2 = Zalgo(v + '#' + u),
        z3 = Zalgo(ru + '#' + rv), z4 = Zalgo(v);
    auto get_z = [](const vector<int> &z, int i) {
        return (0 <= i and i < (int)z.size()) ? z[i] : 0;
    };
    auto add_rep
        = [&](bool left, int c, int l, int k1, int k2) {
        const int L = max(1, l - k2), R = min(l - left, k1);
        if (L > R) return;
        if (left
            ) rep[l].emplace_back(sft + c - R, sft + c - L);
        else rep[l].emplace_back
            (sft + c - R - l + 1, sft + c - L - l + 1);
    };
    FOR (cntr, 0, n - 1) {
        int l, k1, k2;
        if (cntr < nu) {
            l = nu - cntr;
            k1 = get_z(z1, nu - cntr);
            k2 = get_z(z2, nv + 1 + cntr);
        } else {
            l = cntr - nu + 1;
            k1 = get_z(z3, nu + 1 + nv - 1 - (cntr - nu));
            k2 = get_z(z4, (cntr - nu) + 1);
        }
        if (k1 + k2 >= l)
            add_rep(cntr < nu, cntr, l, k1, k2);
    }
} // p \in [l, r] => s[p, p + i) = s[p + i, p + 2i)

```

6 Math

6.1 ax+by=gcd(only exgcd *) [42ad36]

```

typedef pair<ll, ll> pll;
pll exgcd(ll a, ll b) {
    if (b == 0) return pll(1, 0);
    ll p = a / b;
    pll q = exgcd(b, a % b);
    return pll(q.S, q.F - q.S * p);
}
/* ax+by=res, let x be minimum non-negative
g, p = gcd(a, b), exgcd(a, b) * res / g
if p.F < 0: t = (abs(p.F) + b / g - 1) / (b / g)
else: t = -(p.F / (b / g))
p += (b / g, -a / g) * t */

```

6.2 Floor and Ceil [692c04]

```

int floor(int a, int b)
{ return a / b - (a % b && (a < 0) ^ (b < 0)); }
int ceil(int a, int b)
{ return a / b + (a % b && (a < 0) ^ (b > 0)); }

```

6.3 Floor Enumeration [7cbcdf]

```

// enumerating x = floor(n / i), [l, r]
for (int l = 1, r; l <= n; l = r + 1) {
    int x = n / l;
    r = n / x;
}

```

6.4 Mod Min [59ba59]

```

// min{k | l <= ((ak) mod m) <= r}, no solution -> -1
ll mod_min(ll a, ll m, ll l, ll r) {
    if (a == 0) return l ? -1 : 0;
    if (ll k = (l + a - 1) / a; k * a <= r) return k;
    ll b = m / a, c = m % a;
    if (ll y = mod_min(c, a, a - r % a, a - l % a); ~y)
        return (l + y * c + a - 1) / a + y * b;
    return -1;
}

```

6.5 Gaussian integer gcd [0e7740]

```

cpx gaussian_gcd(cpx a, cpx b) {
#define
    rnd(a, b) ((a >= 0 ? a * 2 + b : a * 2 - b) / (b * 2))
    ll c = a.real() * b.real() + a.imag() * b.imag();
    ll d = a.imag() * b.real() - a.real() * b.imag();
    ll r = b.real() * b.real() + b.imag() * b.imag();
    if (c % r == 0 && d % r == 0) return b;
    return
        gaussian_gcd(b, a - cpx(rnd(c, r), rnd(d, r)) * b);
}

```

6.6 Miller Rabin* [0498b2]

```
// n < 4,759,123,141      3 : 2, 7, 61
// n < 1,122,004,669,633 4 : 2, 13, 23, 1662803
// n < 3,474,749,660,383 6 : primes <= 13
// n < 2^64              7 :
// 2, 325, 9375, 28178, 450775, 9780504, 1795265022
bool Miller_Rabin(ll a, ll n) {
    if ((a = a % n) == 0) return 1;
    if (n % 2 == 0) return n == 2;
    ll tmp = (n - 1) / ((n - 1) & (1 - n));
    ll t = __lg(((n - 1) & (1 - n))), x = 1;
    for (; tmp; tmp >>= 1, a = (__int128)a * a % n)
        if (tmp & 1) x = (__int128)x * a % n;
    if (x == 1 || x == n - 1) return 1;
    while (--t)
        if ((x = (__int128)x * x % n) == n - 1) return 1;
    return 0;
}
```

6.7 System of Linear Equations [11f4ff]

```
struct matrix { // m variables, n equations
    int n, m;
    fraction M[MAXN][MAXN + 1], sol[MAXN + 1],
        basis[MAXN][MAXN + 1];
    bool with_basis = true;
    int rank = -1;
    bool fixed[MAXN];
    int solve() { // -1: inconsistent, >= 0: rank of
        // solution space
        FOR (i, 0, n - 1) {
            int piv = 0;
            while (piv < m && !M[i][piv].n) ++piv;
            if (piv == m) continue;
            FOR (j, 0, n - 1) {
                if (i == j) continue;
                fraction tmp = -M[j][piv] / M[i][piv];
                FOR (k, 0, m)
                    M[j][k] = tmp * M[i][k] + M[j][k];
            }
        }
        rank = m;
        FOR (i, 0, n - 1) {
            int piv = 0;
            while (piv < m && !M[i][piv].n) ++piv;
            if (piv == m && M[i][m].n) return rank = -1;
            else if (piv < m) {
                --rank;
                if (with_basis) {
                    for (int j = m; j >= piv; j--)
                        M[i][j] = M[i][j] / M[i][piv];
                    sol[piv] = M[i][m];
                    fixed[piv] = true;
                } else {
                    sol[piv] = M[i][m] / M[i][piv];
                }
            }
        }
        if (with_basis) {
            FOR (i, 0, n - 1) {
                int piv = 0;
                while (piv < m && !M[i][piv].n) ++piv;
                for (int j = 0, k = 0; j < m; j++) {
                    if (!fixed[j]) basis[k++][piv] = -M[i][j];
                }
                for (int j = 0, k = 0; j < m; j++) {
                    if (!fixed[j]) basis[k++][j] = 1;
                }
            }
        }
        return rank;
    }
};
```

6.8 Pollard Rho* [fffd1c]

```
map<ll, int> cnt;
void PollardRho(ll n) {
    if (n == 1) return;
    if (prime(n)) return ++cnt[n], void();
    if (n % 2 == 0) return PollardRho(n / 2), ++cnt[2], void();
    ll x = 2, y = 2, d = 1, p = 1;
    #define f(x, n, p) (((__int128)x * x + p) % n)
    while (true) {
        if (d != n && d != 1) {
            PollardRho(n / d);
            PollardRho(d);
        }
    }
```

```
        return;
    }
    if (d == n) ++p;
    x = f(x, n, p), y = f(f(y, n, p), n, p);
    d = gcd(abs(x - y), n);
}
```

6.9 Simplex Algorithm [76b226]

```
const int MAXN = 11000, MAXM = 405;
const double eps = 1E-10;
double a[MAXN][MAXM], b[MAXN], c[MAXN];
double d[MAXN][MAXM], x[MAXN];
int ix[MAXN + MAXM]; // !!! array all indexed from 0
// max{cx} subject to {Ax<=b, x>=0}
// n: constraints, m: vars !!!
// x[] is the optimal solution vector
// usage :
// value = simplex(a, b, c, N, M);
double simplex(int n, int m) {
    ++m;
    fill_n(d[n], m + 1, 0);
    fill_n(d[n + 1], m + 1, 0);
    iota(ix, ix + n + m, 0);
    int r = n, s = m - 1;
    FOR (i, 0, n - 1) {
        FOR (j, 0, m - 2) d[i][j] = -a[i][j];
        d[i][m - 1] = 1;
        d[i][m] = b[i];
        if (d[r][m] > d[i][m]) r = i;
    }
    copy_n(c, m - 1, d[n]);
    d[n + 1][m - 1] = -1;
    for (double dd; ) {
        if (r < n) {
            swap(ix[s], ix[r + m]);
            d[r][s] = 1.0 / d[r][s];
            FOR (j, 0, m)
                if (j != s) d[r][j] *= -d[r][s];
            FOR (i, 0, n + 1)
                if (i != r) {
                    FOR (j, 0, m)
                        if (j != s) d[i][j] += d[r][j] * d[i][s];
                    d[i][s] *= d[r][s];
                }
        }
        r = s = -1;
        FOR (j, 0, m - 1)
            if (s < 0 || ix[s] > ix[j]) {
                if (d[n + 1][j] > eps ||
                    (d[n + 1][j] > -eps && d[n][j] > eps))
                    s = j;
            }
        if (s < 0) break;
        FOR (i, 0, n - 1)
            if (d[i][s] < -eps) {
                if (r < 0 ||
                    (dd = d[r][m] / d[r][s] -
                     d[i][m] / d[i][s]) < -eps ||
                    (dd < eps && ix[r + m] > ix[i + m]))
                    r = i;
            }
        if (r < 0) return -1; // not bounded
    }
    if (d[n + 1][m] < -eps) return -1; // not executable
    double ans = 0;
    fill_n(x, m, 0);
    FOR (i, m,
        n + m - 1) { // the missing enumerated x[i] = 0
        if (ix[i] < m - 1) {
            ans += d[i - m][m] * c[ix[i]];
            x[ix[i]] = d[i - m][m];
        }
    }
    return ans;
}
```

6.9.1 Construction

Primal	Dual
Maximize $c^T x$ s.t. $Ax \leq b, x \geq 0$	Minimize $b^T y$ s.t. $A^T y \geq c, y \geq 0$
Maximize $c^T x$ s.t. $Ax \leq b$	Minimize $b^T y$ s.t. $A^T y = c, y \geq 0$
Maximize $c^T x$ s.t. $Ax = b, x \geq 0$	Minimize $b^T y$ s.t. $A^T y \geq c$

$\bar{x}$ and $\bar{y}$ are optimal if and only if for all $i \in [1, n]$, either $\bar{x}_i = 0$ or $\sum_{j=1}^m A_{ji} \bar{y}_j = c_i$ holds and for all $i \in [1, m]$ either $\bar{y}_i = 0$ or $\sum_{j=1}^n A_{ij} \bar{x}_j = b_j$ holds.

- In case of minimization, let $c'_i = -c_i$
- $\sum_{1 \leq i \leq n} A_{ji} x_i \geq b_j \rightarrow \sum_{1 \leq i \leq n} -A_{ji} x_i \leq -b_j$
- $\sum_{1 \leq i \leq n} A_{ji} x_i = b_j$

- $\sum_{1 \leq i \leq n} A_{ji} x_i \leq b_j$
- $\sum_{1 \leq i \leq n} A_{ji} x_i \geq b_j$

4. If x_i has no lower bound, replace x_i with $x_i - x'_i$

6.10 chineseRemainder [9e542c]

```
ll solve(ll x1, ll m1, ll x2, ll m2) {
    ll g = gcd(m1, m2);
    if ((x2 - x1) % g) return -1; // no sol
    m1 /= g; m2 /= g;
    pair<ll, ll> p = exgcd(m1, m2);
    ll lcm = m1 * m2 * g;
    ll res = p.F * (x2 - x1) * m1 + x1;
    // be careful with overflow
    return (res % lcm + lcm) % lcm;
}
```

6.11 Factorial without prime factor* [8d24f6]

```
// O(p^k + log^2 n), pk = p^k
ll prod[MAXP];
ll fac_no_p(ll n, ll p, ll pk) {
    prod[0] = 1;
    FOR (i, 1, pk)
        if (i % p) prod[i] = prod[i - 1] * i % pk;
        else prod[i] = prod[i - 1];
    ll rt = 1;
    for (; n; n /= p) {
        rt = rt * mpow(prod[pk], n / pk, pk) % pk;
        rt = rt * prod[n % pk] % pk;
    }
    return rt;
} // (n! without factor p) % p^k
```

6.12 QuadraticResidue* [e0bf30]

```
int Jacobi(int a, int m) {
    int s = 1;
    for (; m > 1; ) {
        a %= m;
        if (a == 0) return 0;
        const int r = __builtin_ctz(a);
        if ((r & 1) && ((m & 2) & 4)) s = -s;
        a >>= r;
        if (a & m & 2) s = -s;
        swap(a, m);
    }
    return s;
}

int QuadraticResidue(int a, int p) {
    if (p == 2) return a & 1;
    const int jc = Jacobi(a, p);
    if (jc == 0) return 0;
    if (jc == -1) return -1;
    int b, d;
    for (; ; ) {
        b = rand() % p;
        d = (1LL * b * b + p - a) % p;
        if (Jacobi(d, p) == -1) break;
    }
    int f0 = b, f1 = 1, g0 = 1, g1 = 0, tmp;
    for (int e = (1LL + p) >> 1; e; e >>= 1) {
        if (e & 1) {
            tmp = (1LL
                * g0 * f0 + 1LL * d * (1LL * g1 * f1 % p)) % p;
            g1 = (1LL * g0 * f1 + 1LL * g1 * f0) % p;
            g0 = tmp;
        }
        tmp = (1
            LL * f0 * f0 + 1LL * d * (1LL * f1 * f1 % p)) % p;
        f1 = (2LL * f0 * f1) % p;
        f0 = tmp;
    }
    return g0;
}
```

6.13 PiCount* [b791a0]

```
ll PrimeCount(ll n) { // n ~ 10^13 => < 2s
    if (n <= 1) return 0;
    int v = sqrt(n), s = (v + 1) / 2, pc = 0;
    vector<int> smalls(v + 1), skip(v + 1), roughs(s);
    vector<ll> larges(s);
    FOR (i, 2, v) smalls[i] = (i + 1) / 2;
    FOR (i, 0, s - 1) {
        roughs[i] = 2 * i + 1;
        larges[i] = (n / (2 * i + 1) + 1) / 2;
    }
    FOR (p, 3, v) {
```

```
if (smalls[p] > smalls[p - 1]) {
    int q = p * p;
    ++pc;
    if (1LL * q * q > n) break;
    skip[p] = 1;
    for (int i = q; i <= v; i += 2 * p) skip[i] = 1;
    int ns = 0;
    FOR (k, 0, s - 1) {
        int i = roughs[k];
        if (skip[i]) continue;
        ll d = 1LL * i * p;
        larges[ns] = larges[k] - (d <= v ?
            larges[smalls[d] - pc] : smalls[n / d]) + pc;
        roughs[ns++] = i;
    }
    s = ns;
    for (int j = v / p; j >= p; j--) {
        int c = smalls[j] - pc, e = min(j * p + p, v + 1);
        FOR (i, j * p, e - 1) smalls[i] -= c;
    }
}
FOR (k, 1, s - 1) {
    const ll m = n / roughs[k];
    ll t = larges[k] - (pc + k - 1);
    FOR (l, 1, k - 1) {
        int p = roughs[l];
        if (1LL * p * p > m) break;
        t -= smalls[m / p] - (pc + l - 1);
    }
    larges[0] -= t;
}
return larges[0];
}
```

6.14 Discrete Log* [7dde1b]

```
int DiscreteLog(int s, int x, int y, int m) {
    int kStep = sqrt(m) + 1;
    unordered_map<int, int> p;
    int b = 1;
    FOR (i, 0, kStep - 1) {
        p[y] = i;
        y = 1LL * y * x % m;
        b = 1LL * b * x % m;
    }
    for (ll i = 0; i <= m; i += kStep) {
        s = 1LL * s * b % m;
        if (p.find(s) != p.end()) return i + kStep - p[s];
    }
    return -1;
}

int DiscreteLog(int x, int y, int m) {
    if (m == 1) return 0;
    int s = 1;
    FOR (i, 0, 99) {
        if (s == y) return i;
        s = 1LL * s * x % m;
    }
    if (s == y) return 100;
    int p = 100 + DiscreteLog(s, x, y, m);
    if (fpow(x, p, m) != y) return -1;
    return p;
} // find minimum k such that x^k == y (mod m)
```

6.15 Berlekamp Massey [d0dd49]

```
#define SZ(a) ((int)a.size())
// a_i = \sum_{j=1}^k c_j a_{i-j}; returns c 1-based,
// c[0] unused, k = SZ(c) - 1. Feeds LinearRecursion.
template <typename T>
vector<T> BerlekampMassey(const vector<T> &output) {
    vector<T> d(SZ(output) + 1), me, he;
    for (int f = 0, i = 1; i <= SZ(output); i++) {
        FOR (j, 0, SZ(me) - 1)
            d[i] += output[i - j - 2] * me[j];
        if ((d[i] -= output[i - 1]) == 0) continue;
        if (me.empty()) {
            me.resize(f = i);
            continue;
        }
        vector<T> o(i - f - 1);
        T k = -d[i] / d[f]; o.pb(-k);
        for (T x : he) o.pb(x * k);
        o.resize(max(SZ(o), SZ(me)));
        FOR (j, 0, SZ(me) - 1) o[j] += me[j];
        if (i - f + SZ(he) >= SZ(me)) he = me, f = i;
        me = o;
    }
}
```

```

me.insert(me.begin(), T()); // me[j] was c_(j+1)
return me;
}

```

6.16 Primes

```

/* 12721 13331 14341 75577 123457 222557 556679 999983
* 1097774749 1076767633 100102021 999997771 1001010013
* 1000512343 987654361 999991231 99988733 98789101
* 987777733 999991921 1010101333 1010102101
* 1000000000039 1000000000000037 2305843009213693951
* 4611686018427387847 9223372036854775783
* 18446744073709551557 */

```

6.17 Theorem

- Cramer's rule

$$\begin{aligned} ax+by &= e & x &= \frac{ed-bf}{ad-bc} \\ cx+dy &= f & y &= \frac{af-ec}{ad-bc} \end{aligned}$$

- Vandermonde's Identity

$$C(n+m, k) = \sum_{i=0}^k C(n, i) C(m, k-i)$$

- Kirchhoff's Theorem

Denote L be a $n \times n$ matrix as the Laplacian matrix of graph G , where $L_{ii} = d(i)$ (in-degree for directed case), $L_{ij} = -c$ where c is the number of edge (i, j) in G .

- The number of undirected spanning in G is $|\det(\tilde{L}_{11})|$.
- The number of directed spanning tree rooted at r in G is $|\det(\tilde{L}_{rr})|$.

- Tutte's Matrix

Let D be a $n \times n$ matrix, where $d_{ij} = x_{ij}$ (x_{ij} is chosen uniformly at random) if $i < j$ and $(i, j) \in E$, otherwise $d_{ij} = -d_{ji}$. $\frac{\text{rank}(D)}{2}$ is the maximum matching on G .

- Cayley's Formula

- Given a degree sequence $d_1, d_2, \dots, d_n$ for each labeled vertices, there are $\frac{(n-2)!}{(d_1-1)!(d_2-1)!\dots(d_n-1)!}$ spanning trees.

- Let $T_{n,k}$ be the number of labeled forests on n vertices with k components, such that vertex $1, 2, \dots, k$ belong to different components. Then $T_{n,k} = kn^{n-k-1}$.

- Erdős-Gallai theorem

A sequence of nonnegative integers $d_1 \geq \dots \geq d_n$ can be represented as the degree sequence of a finite simple graph on n vertices if and only if $d_1 + \dots + d_n$

is even and $\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$ holds for every $1 \leq k \leq n$.

- Gale-Ryser theorem

A pair of sequences of nonnegative integers $a_1 \geq \dots \geq a_n$ and $b_1, \dots, b_n$ is

bigraphic if and only if $\sum_{i=1}^n a_i = \sum_{i=1}^n b_i$ and $\sum_{i=1}^k a_i \leq \sum_{i=1}^n \min(b_i, k)$ holds for every $1 \leq k \leq n$.

- Fulkerson-Chen-Anstee theorem

A sequence $(a_1, b_1), \dots, (a_n, b_n)$ of nonnegative integer pairs

with $a_1 \geq \dots \geq a_n$ is digraphic if and only if $\sum_{i=1}^n a_i = \sum_{i=1}^n b_i$ and

$\sum_{i=1}^k a_i \leq \sum_{i=1}^n \min(b_i, k-1) + \sum_{i=k+1}^n \min(b_i, k)$ holds for every $1 \leq k \leq n$.

- Pick's theorem

For simple polygon, when points are all integer, we have $A = \#\{\text{lattice points in the interior}\} + \frac{\#\{\text{lattice points on the boundary}\}}{2} - 1$.

- Möbius inversion formula

- $f(n) = \sum_{d|n} g(d) \Leftrightarrow g(n) = \sum_{d|n} \mu(d) f(\frac{n}{d})$
- $f(n) = \sum_{n|d} g(d) \Leftrightarrow g(n) = \sum_{n|d} \mu(\frac{d}{n}) f(d)$

- Spherical cap

- A portion of a sphere cut off by a plane.
- r : sphere radius, a : radius of the base of the cap, h : height of the cap, θ : $\arcsin(a/r)$.
- Volume $= \pi h^2 (3r-h)/3 = \pi h (3a^2 + h^2)/6 = \pi r^3 (2 + \cos\theta)(1 - \cos\theta)^2/3$.
- Area $= 2\pi r h = \pi(a^2 + h^2) = 2\pi r^2 (1 - \cos\theta)$.

- Lagrange multiplier

- Optimize $f(x_1, \dots, x_n)$ when k constraints $g_i(x_1, \dots, x_n) = 0$.
- Lagrangian function $\mathcal{L}(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k) = f(x_1, \dots, x_n) - \sum_{i=1}^k \lambda_i g_i(x_1, \dots, x_n)$.
- The solution corresponding to the original constrained optimization is always a saddle point of the Lagrangian function.

- Nearest points of two skew lines

- Line 1: $v_1 = p_1 + t_1 d_1$
- Line 2: $v_2 = p_2 + t_2 d_2$
- $n = d_1 \times d_2$
- $n_1 = d_1 \times n$
- $n_2 = d_2 \times n$

$$\begin{aligned} c_1 &= p_1 + \frac{(p_2 - p_1) \cdot n_2}{d_1 \cdot n_2} d_1 \\ c_2 &= p_2 + \frac{(p_1 - p_2) \cdot n_1}{d_2 \cdot n_1} d_2 \end{aligned}$$

- Derivatives/Integrals

$$\begin{aligned} \text{Integration by parts: } \int_a^b f(x)g(x)dx &= [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx \\ \left| \frac{d}{dx} \sin^{-1}x = \frac{1}{\sqrt{1-x^2}} \right| & \left| \frac{d}{dx} \cos^{-1}x = -\frac{1}{\sqrt{1-x^2}} \right| & \left| \frac{d}{dx} \tan^{-1}x = \frac{1}{1+x^2} \right| \\ \left| \frac{d}{dx} \tan x = 1 + \tan^2 x \right| & \left| \int \tan ax = -\frac{\ln|\cos ax|}{a} \right| \\ \left| \int e^{-x^2} = \frac{\sqrt{\pi}}{2} \text{erf}(x) \right| & \left| \int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax-1) \right| \\ \left| \int \sqrt{a^2+x^2} = \frac{1}{2} (x\sqrt{a^2+x^2} + a^2 \sinh(x/a)) \right| \end{aligned}$$

- Spherical Coordinate

$$(x, y, z) = (r \sin\theta \cos\phi, r \sin\theta \sin\phi, r \cos\theta)$$

$$(r, \theta, \phi) = (\sqrt{x^2 + y^2 + z^2}, \arccos(z/\sqrt{x^2 + y^2 + z^2}), \text{atan2}(y, x))$$

- Rotation Matrix

$$M(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, R_x(\theta_x) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta_x & -\sin\theta_x \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

- Min-Max Inclusion Exclusion Principle

(Works in expect values version, i.e., $E[\min]$, too)

$$\text{kthmax}_{i \in S} x_i = \sum_{T \subset S} (-1)^{|T|-k} \binom{|T|-1}{k-1} \min_{j \in T} x_j$$

$$\text{kthmin}_{i \in S} x_i = \sum_{T \subset S} (-1)^{|T|-k} \binom{|T|-1}{k-1} \max_{j \in T} x_j$$

6.18 Estimation

n	2	3	4	5	6	7	8	9	20	30	40	50	100		
$p(n)$	2	3	5	7	11	15	22	30	627	5604	4e4	2e5	2e8		
n	100	1e3	1e6	1e9	1e12	1e15	1e18								
$d(i)$	12	32	240	1344	6720	26880	103680								
n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\binom{2n}{n}$	2	6	20	70	252	924	3432	12870	48620	184756	7e5	2e6	1e7	4e7	1.5e8
n	2	3	4	5	6	7	8	9	10	11	12	13			
B_n	2	5	15	52	203	877	4140	21147	115975	7e5	4e6	3e7			

6.19 Euclidean Algorithms

- $m = \lfloor \frac{a+b}{c} \rfloor$
- Time complexity: $O(\log n)$

$$\begin{aligned} f(a, b, c, n) &= \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor \\ &= \begin{cases} \lfloor \frac{a}{c} \rfloor \cdot \frac{n(n+1)}{2} + \lfloor \frac{b}{c} \rfloor \cdot (n+1) \\ + f(a \bmod c, b \bmod c, c, n), & a \geq c \vee b \geq c \\ 0, & n < 0 \vee a = 0 \\ nm - f(c, c-b-1, a, m-1), & \text{otherwise} \end{cases} \end{aligned}$$

$$\begin{aligned} g(a, b, c, n) &= \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor \\ &= \begin{cases} \lfloor \frac{a}{c} \rfloor \cdot \frac{n(n+1)(2n+1)}{6} + \lfloor \frac{b}{c} \rfloor \cdot \frac{n(n+1)}{2} \\ + g(a \bmod c, b \bmod c, c, n), & a \geq c \vee b \geq c \\ 0, & n < 0 \vee a = 0 \\ \frac{1}{2} \cdot (n(n+1)m - f(c, c-b-1, a, m-1)) \\ - h(c, c-b-1, a, m-1), & \text{otherwise} \end{cases} \end{aligned}$$

$$\begin{aligned} h(a, b, c, n) &= \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2 \\ &= \begin{cases} \lfloor \frac{a}{c} \rfloor^2 \cdot \frac{n(n+1)(2n+1)}{6} + \lfloor \frac{b}{c} \rfloor^2 \cdot (n+1) \\ + \lfloor \frac{a}{c} \rfloor \cdot \lfloor \frac{b}{c} \rfloor \cdot n(n+1) \\ + h(a \bmod c, b \bmod c, c, n) \\ + 2 \lfloor \frac{a}{c} \rfloor \cdot g(a \bmod c, b \bmod c, c, n) \\ + 2 \lfloor \frac{b}{c} \rfloor \cdot f(a \bmod c, b \bmod c, c, n), & a \geq c \vee b \geq c \\ 0, & n < 0 \vee a = 0 \\ nm(m+1) - 2g(c, c-b-1, a, m-1) \\ - 2f(c, c-b-1, a, m-1) - f(a, b, c, n), & \text{otherwise} \end{cases} \end{aligned}$$

6.20 General Purpose Numbers

- Bernoulli numbers

$$B_0 = 1, B_1^\pm = \pm \frac{1}{2}, B_2 = \frac{1}{6}, B_3 = 0$$

$$\sum_{j=0}^m \binom{m+1}{j} B_j = 0, \text{EGF is } B(x) = \frac{x}{e^x - 1} = \sum_{n=0}^{\infty} B_n \frac{x^n}{n!}.$$

$$S_m(n) = \sum_{k=1}^n k^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k^+ n^{m+1-k}$$

- Stirling numbers of the second kind Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n-1, k-1) + kS(n-1, k), S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$$

$$x^n = \sum_{i=0}^n S(n, i) (x)_i$$

- Pentagonal number theorem

$$\prod_{n=1}^{\infty} (1-x^n) = 1 + \sum_{k=1}^{\infty} (-1)^k \left(x^{k(3k+1)/2} + x^{k(3k-1)/2} \right)$$

- Catalan numbers

$$C_n^{(k)} = \frac{1}{(k-1)n+1} \binom{kn}{n}$$

$$C^{(k)}(x) = 1 + x[C^{(k)}(x)]^k$$

- Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k, j : s.t. $\pi(j) > \pi(j+1)$, $k+1, j$: s.t. $\pi(j) \geq j$, k, j : s.t. $\pi(j) > j$.

$$E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$$

$$E(n, 0) = E(n, n-1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.21 Tips for Generating Functions

- Ordinary Generating Function $A(x) = \sum_{i \geq 0} a_i x^i$

$$A(rx) \Rightarrow r^n a_n$$

$$A(x) + B(x) \Rightarrow a_n + b_n$$

$$A(x)B(x) \Rightarrow \sum_{i=0}^n a_i b_{n-i}$$

$$A(x)^k \Rightarrow \sum_{i_1+i_2+\dots+i_k=n} a_{i_1} a_{i_2} \dots a_{i_k}$$

$$xA(x)' \Rightarrow na_n$$

$$\frac{A(x)}{1-x} \Rightarrow \sum_{i=0}^n a_i$$

- Exponential Generating Function $A(x) = \sum_{i \geq 0} \frac{a_i}{i!} x^i$

$$A(x) + B(x) \Rightarrow a_n + b_n$$

$$A^{(k)}(x) \Rightarrow a_{n+k}$$

$$A(x)B(x) \Rightarrow \sum_{i=0}^n \binom{n}{i} a_i b_{n-i}$$

$$A(x)^k \Rightarrow \sum_{i_1+i_2+\dots+i_k=n} \binom{n}{i_1, i_2, \dots, i_k} a_{i_1} a_{i_2} \dots a_{i_k}$$

$$xA(x) \Rightarrow na_n$$

- Special Generating Function

$$(1+x)^n = \sum_{i \geq 0} \binom{n}{i} x^i$$

$$\frac{1}{(1-x)^n} = \sum_{i \geq 0} \binom{n-1}{i} x^i$$

7 Polynomial

7.1 Fast Fourier Transform [a07ad9]

```
typedef complex<double> cplx;
// safe if (sum a_i^2 + sum b_i^2) log2(n) < 9e14
void fft(vector<cplx> &a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<cplx> rt(2, 1);
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n), rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        for (i, k, 2 * k - 1)
            rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
    }
    vector<int> rev(n);
    for (i, 0, n - 1)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (i, 0, n - 1)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (j, 0, k - 1) {
                cplx z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
}

vector<double> conv(
    const vector<double> &a, const vector<double> &b) {
    if (a.empty() || b.empty()) return {};
    int m = a.size() + b.size() - 1;
    int L = 32 - __builtin_clz(m), n = 1 << L;
    vector<double> res(m);
    vector<cplx> in(n), out(n);
    copy(a.begin(), a.end(), in.begin());
    for (i, 0, (int)b.size() - 1) in[i].imag(b[i]);
    fft(in);
    for (cplx &x : in) x *= x;
    for (i, 0, n - 1)
        out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    for (i, 0, m - 1) res[i] = imag(out[i]) / (4 * n);
    return res;
}
```

7.2 FFT Arbitrary Mod [5776da]

```
// needs FFT; ok while n log2(n) * M < 8.6e14
// inputs must already be in [0, M)
template <int M>
vector<ll> conv_mod(
    const vector<ll> &a, const vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int m = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(m), n = 1 << B;
    int cut = (int)sqrt(M);
    vector<ll> res(m);
    vector<cplx> L(n), R(n), os(n), ol(n);
    for (i, 0, (int)a.size() - 1)
        L[i] = cplx((int)a[i] / cut, (int)a[i] % cut);
    for (i, 0, (int)b.size() - 1)
        R[i] = cplx((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    for (i, 0, n - 1) {
        int j = -i & (n - 1);
        ol[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        os[j] =
            (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(ol), fft(os);
    for (i, 0, m - 1) {
        ll av = (ll)(real(ol[i]) + .5);
        ll cv = (ll)(imag(os[i]) + .5);
        ll bv =
            (ll)(imag(ol[i]) + .5) + (ll)(real(os[i]) + .5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

7.3 Number Theory Transform* [0711de]

```
//(2^16)+1, 65537, 3
// 7*17*(2^23)+1, 998244353, 3
// 1255*(2^20)+1, 1315962881, 3
// 51*(2^25)+1, 1711276033, 29
template <int MAXN, int P, int RT> // MAXN must be 2^k
struct NTT {
    int w[MAXN];
    int mpow(int a, ll n);
    int minv(int a) { return mpow(a, P - 2); }
    NTT() {
        int dw = mpow(RT, (P - 1) / MAXN);
        w[0] = 1;
        for (i, 1, MAXN - 1)
            w[i] = (ll)w[i - 1] * dw % P;
    }
    void bitrev(int *a, int n) {
        int i = 0;
        for (j, 1, n - 2) {
            for (int k = n >> 1; (i ^ k) < k; k >>= 1);
            if (j < i) swap(a[i], a[j]);
        }
    }
    void operator()(
        int *a, int n, bool inv = false) { // 0 <= a[i] < P
        bitrev(a, n);
        for (int L = 2; L <= n; L <= 1) {
            int dx = MAXN / L, dl = L >> 1;
            for (int i = 0; i < n; i += L) {
                for (int j = i, x = 0; j < i + dl;
                    j++, x += dx) {
                    int tmp = (ll)a[j + dl] * w[x] % P;
                    if ((a[j + dl] = a[j] - tmp) < 0)
                        a[j + dl] += P;
                    if ((a[j] += tmp) >= P) a[j] -= P;
                }
            }
        }
        if (inv) {
            reverse(a + 1, a + n);
            int invn = minv(n);
            for (i, 0, n - 1) a[i] = (ll)a[i] * invn % P;
        }
    }
};
```

7.4 NTT Primes

$P = c \cdot 2^k + 1$ with primitive root g ; 2^k bounds the transform length.

P	g	2^k	P	g	2^k
7681	17	2^9	104857601	3	2^{22}
12289	11	2^{12}	985661441	3	2^{22}
40961	3	2^{13}	998244353	3	2^{23}
65537	3	2^{16}	2810183681	11	2^{23}
786433	10	2^{18}	167772161	3	2^{25}
5767169	3	2^{19}	1107296257	10	2^{25}
7340033	3	2^{20}	469762049	3	2^{26}
605028353	3	2^{20}	2885681153	3	2^{26}
23068673	3	2^{21}	2013265921	31	2^{27}

2061584302081	7	2^{37}
2748779069441	3	2^{39}
9223372036737335297	3	2^{24}
194555039024054273	5	2^{56}

7.5 Fast Walsh Transform* [c5d033]

```

/* x: a[j], y: a[j + (L >> 1)]
or: (y += x * op), and: (x += y * op)
xor: (x, y = (x + y) * op, (x - y) * op)
invop: or, and, xor = -1, -1, 1/2 */
void fwt(int *a, int n, int op) { //or
    for (int L = 2; L <= n; L <= 1)
        for (int i = 0; i < n; i += L)
            FOR (j, i, j + (L >> 1) - 1)
                a[j + (L >> 1)] += a[j] * op;
}

const int N = 21;
int f[N][1 << N], g[N][1 << N], h[N][1 << N], ct[1 << N];
void subset_convolution(int *a, int *b, int *c, int L) {
    // c_k = \sum_{i+j=k, i&j=0} a_i * b_j
    int n = 1 << L;
    FOR (i, 1, n - 1)
        ct[i] = ct[i & (i - 1)] + 1;
    FOR (i, 0, n - 1)
        f[ct[i]][i] = a[i], g[ct[i]][i] = b[i];
    FOR (i, 0, L)
        fwt(f[i], n, 1), fwt(g[i], n, 1);
    FOR (i, 0, L) FOR (j, 0, i) FOR (x, 0, n - 1)
        h[i][x] += f[j][x] * g[i - j][x];
    FOR (i, 0, L)
        fwt(h[i], n, -1);
    FOR (i, 0, n - 1)
        c[i] = h[ct[i]][i];
}

```

7.6 Polynomial Operation [f3c9b0]

```

template<int MAXN, int P, int RT> // MAXN = 2^k
struct Poly : vector<int> { // coefficients in [0, P)
    using vector<int>::vector;
    static NTT<MAXN, P, RT> ntt;
    int n() const { return (int)size(); } // n() >= 1
    Poly(const Poly &p, int m) : vector<int>(m) {
        copy_n(p.data(), min(p.n(), m), data());
    }
    Poly& irev
        () { return reverse(data(), data() + n()), *this; }
    Poly& isz(int m) { return resize(m), *this; }
    Poly& iadd(const Poly &rhs) { // n() == rhs.n()
        FOR (i, 0, n() - 1)
            if (((*this)[i] += rhs[i]) >= P) (*this)[i] -= P;
        return *this;
    }
    Poly& imul(int k) {
        FOR (i, 0, n() - 1)
            (*this)[i] = (ll)(*this)[i] * k % P;
        return *this;
    }
    Poly Mul(const Poly &rhs) const {
        int m = 1;
        while (m < n() + rhs.n() - 1) m <= 1;
        Poly X(*this, m), Y(rhs, m);
        ntt(X.data(), m), ntt(Y.data(), m);
        FOR (i, 0, m - 1) X[i] = (ll)X[i] * Y[i] % P;
        ntt(X.data(), m, true);
        return X.isz(n() + rhs.n() - 1);
    }
    Poly Inv() const { // (*this)[0] != 0, 1e5/55ms
        if (n() == 1) return {ntt.minv((*this)[0])};
        int m = 1;
        while (m < n() * 2) m <= 1;
        Poly Xi = Poly(*this, (n() + 1) / 2).Inv().isz(m);
        Poly Y(*this, m);
        ntt(Xi.data(), m), ntt(Y.data(), m);
        FOR (i, 0, m - 1) {
            ll t = (2 - (ll)Xi[i] * Y[i]) % P;
            if ((Xi[i] = t * Xi[i] % P) < 0) Xi[i] += P;
        }
    }
}

```

```

ntt(Xi.data(), m, true);
return Xi.isz(n());
}
Poly _sqrt() const { // (*this)[0] is a QR, 1e5/145ms
    if (n() == 1) return {QuadraticResidue((*this)[0], P)};
    Poly X = Poly(*this, (n() + 1) / 2)._sqrt().isz(n());
    return X.iadd(Mul(X.Inv()).isz(n())).imul(P / 2 + 1);
}
Poly Sqrt() const { // {} when no square root exists
    int z = 0;
    while (z < n() && !(*this)[z]) z++;
    if (z == n()) return Poly(n());
    if (z & 1) return {};
    Poly A(data() + z, data() + n());
    if (QuadraticResidue(A[0], P) < 0) return {};
    Poly X = A._sqrt(), R(n());
    FOR (i, 0, X.n() - 1) R[i + z / 2] = X[i];
    return R;
}
pair<Poly, Poly>
    DivMod(const Poly &rhs) const { // (rhs.)back() != 0
    if (n() < rhs.n()) return {{0}, *this};
    const int m = n() - rhs.n() + 1;
    Poly X(rhs); X.irev().isz(m);
    Poly Y(*this); Y.irev().isz(m);
    Poly Q = Y.Mul(X.Inv()).isz(m).irev();
    X = rhs.Mul(Q), Y = *this;
    FOR (i, 0, n() - 1)
        if ((Y[i] -= X[i]) < 0) Y[i] += P;
    return {Q, Y.isz(max(1, rhs.n() - 1))};
}
Poly Dx() const {
    Poly ret(n() - 1);
    FOR (i, 0, ret.n() - 1)
        ret[i] = (ll)(i + 1) * (*this)[i + 1] % P;
    return ret.isz(max(1, ret.n()));
}
Poly Sx() const {
    Poly ret(n() + 1);
    FOR (i, 0, n() - 1)
        ret[i + 1] = (ll)ntt.minv(i + 1) * (*this)[i] % P;
    return ret;
}
Poly _tmul(int nn, const Poly &rhs) const {
    Poly Y = Mul(rhs).isz(n() + nn - 1);
    return Poly(Y.data() + n() - 1, Y.data() + Y.n());
}
vector<int> _eval(const
    vector<int> &x, const vector<Poly> &up) const {
    const int m = (int)x.size();
    if (!m) return {};
    vector<Poly> down(m * 2);
    // down[1] = DivMod(up[1]).S;
    // FOR (i, 2, m * 2 - 1) down[i] = down[i /
    // 2].DivMod(up[i]).S;
    down[1] = Poly(up
        [1]).irev().isz(n()).Inv().irev()._tmul(m, *this);
    FOR (i, 2, m * 2 - 1)
        down[i] =
            up[i ^ 1]._tmul(up[i].n() - 1, down[i / 2]);
    vector<int> y(m);
    FOR (i, 0, m - 1) y[i] = down[m + i][0];
    return y;
}
static vector<Poly> _tree1(const vector<int> &x) {
    const int m = (int)x.size();
    vector<Poly> up(m * 2);
    FOR (i, 0, m - 1)
        up[m + i] = {(x[i] ? P - x[i] : 0), 1};
    for (int i = m - 1;
        i > 0; i--) up[i] = up[i * 2].Mul(up[i * 2 + 1]);
    return up;
}
vector<
    int> Eval(const vector<int> &x) const { // 1e5, 0.6s
    auto up = _tree1(x); return _eval(x, up);
}
static Poly Interpolate(const
    vector<int> &x, const vector<int> &y) { // 1e5, 0.9s
    const int m = (int)x.size();
    vector<Poly> up = _tree1(x), down(m * 2);
    vector<int> z = up[1].Dx()._eval(x, up);
    FOR (
        i, 0, m - 1) z[i] = (ll)y[i] * ntt.minv(z[i]) % P;
    FOR (i, 0, m - 1) down[m + i] = {z[i]};
    for (int i
        = m - 1; i > 0; i--) down[i] = down[i * 2].Mul(up

```

```

    [i * 2 + 1]).iadd(down[i * 2 + 1].Mul(up[i * 2]));
    return down[1];
}
Poly Ln() const { // (*this)[0] == 1, 1e5/100ms
    return Dx().Mul(Inv()).Sx().isz(n());
}
Poly Exp() const { // (*this)[0] == 0, 1e5/215ms
    if (n() == 1) return {1};
    Poly X = Poly(*this, (n() + 1) / 2).Exp().isz(n());
    Poly Y = X.Ln(); Y[0] = P - 1;
    FOR (i, 0, n() - 1)
        if ((Y[i] = (*this)[i] - Y[i]) < 0) Y[i] += P;
    return X.Mul(Y).isz(n());
}
// M := P(P - 1). If k >= M, k := k % M + M.
Poly Pow(ll k) const {
    int nz = 0;
    while (nz < n() && !(*this)[nz]) nz++;
    if (nz * min(k, (ll)n()) >= n()) return Poly(n());
    if (!k) return Poly(Poly{1}, n());
    Poly X(data() + nz, data() + nz + n() - nz * k);
    const int c = ntt.mpow(X[0], k % (P - 1));
    return X.Ln()
        .imul(k % P).Exp().imul(c).irev().isz(n()).irev();
}
static pair<vector<int>, vector<int>> _fac(int m) {
    vector<int> f(m), g(m); // f[i] = i!, g[i] = 1 / i!
    f[0] = 1;
    FOR (i, 1, m - 1) f[i] = (ll)f[i - 1] * i % P;
    g[m - 1] = ntt.minv(f[m - 1]);
    for (int i = m - 1; i > 0; i--)
        g[i - 1] = (ll)g[i] * i % P;
    return {f, g};
}
Poly Shift(ll c) const { // f(x) -> f(x + c)
    auto [f, g] = _fac(n());
    Poly X(*this), Y(n());
    int w = 1, d = (c % P + P) % P;
    FOR (i, 0, n() - 1) {
        X[i] = (ll)X[i] * f[i] % P;
        Y[i] = (ll)g[i] * w % P, w = (ll)w * d % P;
    }
    X = X.irev().Mul(Y).isz(n()).irev();
    FOR (i, 0, n() - 1) X[i] = (ll)X[i] * g[i] % P;
    return X;
}
// f(0), ..., f(k-1) -> f(c), ..., f(c+m-1), deg f < k
static vector<int>
    > SampleShift(const vector<int> &a, ll c, int m) {
    const int k = (int)a.size();
    auto [f, g] = _fac(max(k, m));
    Poly X(k), Y(k);
    FOR (i, 0, k - 1) {
        X[i] = (ll)a[i] * g[i] % P;
        Y[i] = i & 1 ? P - g[i] : g[i];
    }
    X = X.Mul(Y).isz(k);
    FOR (i, 0, k - 1) X[i] = (ll)X[i] * f[i] % P;
    X.irev();
    int w = 1;
    FOR (i, 0, k - 1) {
        Y[i] = (ll)g[i] * w % P;
        w = (ll)w * ((c - i) % P + P) % P;
    }
    X = X.Mul(Y).isz(k).irev();
    FOR (i, 0, k - 1) X[i] = (ll)X[i] * g[i] % P;
    X.isz(m), Y.isz(m);
    FOR (i, 0, m - 1) Y[i] = g[i];
    X = X.Mul(Y).isz(m);
    FOR (i, 0, m - 1) X[i] = (ll)X[i] * f[i] % P;
    return X;
}
// ret[i] = \sum_j w_j [x^j] f^i, f[0] == 0, 2^17/3.2s
Poly PowerProj(vector<int> w, int m) const {
    int k = 1, nn = 1;
    while (nn < n()) nn <= 1;
    const int n2 = nn * 2;
    Poly f(*this, nn), pp(n2), qq(n2);
    w.resize(nn), reverse(w.begin(), w.end());
    FOR (i, 0, nn - 1)
        pp[i] = w[i], qq[i] = f[i] ? P - f[i] : 0;
    while (nn > 1) {
        Poly r(n2);
        FOR (i, 0, n2 - 1)
            r[i] = i & 1 && qq[i] ? P - qq[i] : qq[i];
        Poly a = pp.Mul(r).isz(n2 * 2);
        Poly b = qq.Mul(r).isz(n2 * 2);
        FOR (i, 0, n2 - 1) {

```

```

            a[n2 + i] = ((ll)a[n2 + i] + pp[i]) % P;
            b[n2 + i] = ((ll)b[n2 + i] + qq[i] + r[i]) % P;
        }
        fill(pp.begin(), pp.end(), 0);
        fill(qq.begin(), qq.end(), 0);
        FOR (j, 0, k * 2 - 1) FOR (i, 0, nn / 2 - 1) {
            pp[nn * j + i] = a[nn * j * 2 + i * 2 + 1];
            qq[nn * j + i] = b[nn * j * 2 + i * 2];
        }
        nn /= 2, k *= 2;
    }
    Poly p(k);
    FOR (i, 0, k - 1) p[i] = pp[i * 2];
    return p.irev().isz(m + 1);
}
static int LinearRecursion(const vector<int> &a, const
    vector<int> &coef, ll n) { // a_n = \sum c_j a_{n-j}
    const int k = (int)a.size();
    assert((int)coef.size() == k + 1);
    Poly C(k + 1), W(Poly{1}, k), M = {0, 1};
    FOR (i, 1, k) C[k - i] = coef[i] ? P - coef[i] : 0;
    C[k] = 1;
    // C is fixed: invert reverse(C) once, not inside
    // each DivMod. max(2, k) >= every quotient length
    Poly Ci = Poly(C).irev().isz(max(2, k)).Inv();
    auto rem =
        [&](Poly X) -> Poly { // X mod C, X.n() <= 2k - 1
            if (X.n() < C.n()) return X;
            const int m = X.n() - C.n() + 1;
            Poly Q = Poly(
                X).irev().isz(m).Mul(Poly(Ci, m)).isz(m).irev();
            Poly R = C.Mul(Q);
            FOR (i, 0, X.n() - 1)
                if ((X[i] - R[i]) < 0) X[i] += P;
            return X.isz(k);
        };
    while (n) {
        if (n % 2) W = rem(W.Mul(M));
        n /= 2, M = rem(M.Mul(M));
    }
    ll ret = 0;
    FOR (i, 0, k - 1) ret = (ret + (ll)W[i] * a[i]) % P;
    return ret;
}
};
using Poly_t = Poly<131072 * 2, 998244353, 3>;
template<> decltype(Poly_t::ntt) Poly_t::ntt = {};

```

7.7 Value Polynomial [521fc2]

```

#define SZ(a) ((int)a.size())
struct Poly {
    mint base; // f(x) = poly[x - base]
    vector<mint> poly;
    Poly(mint b = 0, mint x = 0): base(b), poly(1, x) {}
    mint get_val(const mint &x) {
        if (x >= base && x < base + SZ(poly))
            return poly[x - base];
        mint rt = 0;
        vector<mint> lmul(SZ(poly), 1), rmul(SZ(poly), 1);
        FOR (i, 1, SZ(poly) - 1)
            lmul[i] = lmul[i - 1] * (x - (base + i - 1));
        for (int i = SZ(poly) - 2; i >= 0; i--)
            rmul[i] = rmul[i + 1] * (x - (base + i + 1));
        FOR (i, 0, SZ(poly) - 1)
            rt += poly[i] * ifac[i]
                * inegfac[SZ(poly) - 1 - i] * lmul[i] * rmul[i];
        return rt;
    }
    void raise() { // g(x) = \sigma_{base:x} f(x)
        if (SZ(poly) == 1 && poly[0] == 0)
            return;
        mint nw = get_val(base + SZ(poly));
        poly.pb(nw);
        FOR (i, 1, SZ(poly) - 1) poly[i] += poly[i - 1];
    }
};

```

7.8 Newton's Method

Given $F(x)$ where

$$F(x) = \sum_{i=0}^{\infty} \alpha_i (x - \beta)^i$$

for β being some constant. Polynomial P such that $F(P) = 0$ can be found iteratively. Denote by Q_k the polynomial such that $F(Q_k) = 0 \pmod{x^{2^k}}$, then

$$Q_{k+1} = Q_k - \frac{F(Q_k)}{F'(Q_k)} \pmod{x^{2^{k+1}}}$$

8 Geometry

8.1 Default Code [c447c4]

```
#define X first
#define Y second
#define SZ(a) ((int)a.size())
#define ALL(v) v.begin(), v.end()
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef pair<double, double> pdd;
typedef pair<pdd, pdd> Line;
struct Cir{ pdd O; double R; };
const double eps = 1e-8;
pdd operator+(pdd a, pdd b)
{ return pdd(a.X + b.X, a.Y + b.Y); }
pdd operator-(pdd a, pdd b)
{ return pdd(a.X - b.X, a.Y - b.Y); }
pdd operator*(pdd a, double b)
{ return pdd(a.X * b, a.Y * b); }
pdd operator/(pdd a, double b)
{ return pdd(a.X / b, a.Y / b); }
double dot(pdd a, pdd b)
{ return a.X * b.X + a.Y * b.Y; }
double cross(pdd a, pdd b)
{ return a.X * b.Y - a.Y * b.X; }
double abs2(pdd a)
{ return dot(a, a); }
double abs(pdd a)
{ return sqrt(dot(a, a)); }
int sign(double a)
{ return fabs(a) < eps ? 0 : a > 0 ? 1 : -1; }
int ori(pdd a, pdd b, pdd c)
{ return sign(cross(b - a, c - a)); }
bool collinearity(pdd p1, pdd p2, pdd p3)
{ return sign(cross(p1 - p3, p2 - p3)) == 0; }
bool btw(pdd p1, pdd p2, pdd p3) {
    if (!collinearity(p1, p2, p3)) return 0;
    return sign(dot(p1 - p3, p2 - p3)) <= 0;
}
bool seg_intersect(pdd p1, pdd p2, pdd p3, pdd p4) {
    int a123 = ori(p1, p2, p3);
    int a124 = ori(p1, p2, p4);
    int a341 = ori(p3, p4, p1);
    int a342 = ori(p3, p4, p2);
    if (a123 == 0 && a124 == 0)
        return btw(p1, p2, p3) || btw(p1, p2, p4) ||
            btw(p3, p4, p1) || btw(p3, p4, p2);
    return a123 * a124 <= 0 && a341 * a342 <= 0;
}
pdd intersect(pdd p1, pdd p2, pdd p3, pdd p4) {
    double a123 = cross(p2 - p1, p3 - p1);
    double a124 = cross(p2 - p1, p4 - p1);
    return (
        p4 * a123 - p3 * a124) / (a123 - a124); // C^3 / C^2
}
pdd perp(pdd p1)
{ return pdd(-p1.Y, p1.X); }
pdd projection(pdd p1, pdd p2, pdd p3)
{ return p1
    + (p2 - p1) * dot(p3 - p1, p2 - p1) / abs2(p2 - p1); }
pdd reflection(pdd p1, pdd p2, pdd p3)
{ return p3 + perp(p2
    - p1) * cross(p3 - p1, p2 - p1) / abs2(p2 - p1) * 2; }
pdd linearTransformation
    (pdd p0, pdd p1, pdd q0, pdd q1, pdd r) {
    pdd dp = p1
        - p0, dq = q1 - q0, num(cross(dp, dq), dot(dp, dq));
    return q0 + pdd
        (cross(r - p0, num), dot(r - p0, num)) / abs2(dp);
} // from line p0--p1 to q0--q1, apply to r
```

8.2 PointSegDist* [57b6de]

```
double PointSegDist(pdd q0, pdd q1, pdd p) {
    if (sign(abs(q0 - q1)) == 0) return abs(q0 - p);
    if (sign(dot(q1 - q0, p - q0)) >= 0 &&
        sign(dot(q0 - q1, p - q1)) >= 0)
        return fabs(cross(q1 - q0, p - q0) / abs(q0 - q1));
    return min(abs(p - q0), abs(p - q1));
}
```

8.3 Heart [4a0194]

```
pdd circenter
    (pdd p0, pdd p1, pdd p2) { // radius = abs(center)
    pdd center;
    p1 = p1 - p0, p2 = p2 - p0;
    double x1 = p1.X, y1 = p1.Y, x2 = p2.X, y2 = p2.Y;
    double m = 2. * (x1 * y2 - y1 * x2);
```

```
    center.X = (x1
        * x1 * y2 - x2 * x2 * y1 + y1 * y2 * (y1 - y2)) / m;
    center.Y = (x1
        * x2 * (x2 - x1) - y1 * y1 * x2 + x1 * y2 * y2) / m;
    return center + p0;
}
pdd incenter
    (pdd p1, pdd p2, pdd p3) { // radius = area / s * 2
    double a
        = abs(p2 - p3), b = abs(p1 - p3), c = abs(p1 - p2);
    double s = a + b + c;
    return (p1 * a + p2 * b + p3 * c) / s;
}
pdd masscenter(pdd p1, pdd p2, pdd p3)
{ return (p1 + p2 + p3) / 3; }
pdd orthcenter(pdd p1, pdd p2, pdd p3)
{ return masscenter
    (p1, p2, p3) * 3 - circenter(p1, p2, p3) * 2; }
```

8.4 point in circle [a0508f]

```
// return
    q's relation with circumcircle of tri(p[0],p[1],p[2])
bool in_cc(const array<pll, 3> &p, pll q) {
    __int128 det = 0;
    FOR (i, 0, 2)
        det += __int128(abs2(p[i]) - abs2(q)
            ) * cross(p[(i + 1) % 3] - q, p[(i + 2) % 3] - q);
    return det > 0; // in: >0, on: =0, out: <0
}
```

8.5 Convex hull* [b58b5e]

```
void hull(vector<pll> &dots) { // n=1 => ans = {}
    sort(dots.begin(), dots.end());
    vector<pll> ans(1, dots[0]);
    for (int ct = 0; ct < 2; ct++, reverse(ALL(dots)))
        for (int i
            = 1, t = SZ(ans); i < SZ(dots); ans.pb(dots[i++]))
            while (SZ(ans) > t &&
                ori(ans[SZ(ans) - 2], ans.back(), dots[i]) <= 0)
                ans.pop_back();
            ans.pop_back(), ans.swap(dots);
}
```

8.6 PointInConvex* [f86640]

```
bool PointInConvex
    (const vector<pll> &C, pll p, bool strict = true) {
    int a = 1, b = SZ(C) - 1, r = !strict;
    if (SZ(C) == 0) return false;
    if (SZ(C) < 3) return r && btw(C[0], C.back(), p);
    if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
    if (ori(C[0], C[a], p) >= r || ori(C[0], C[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (ori(C[0], C[c], p) > 0 ? b : a) = c;
    }
    return ori(C[a], C[b], p) < r;
}
```

8.7 TangentPointToHull* [523bc1]

```
/* The point should be strictly out of hull
    return arbitrary point on the tangent line */
pii get_tangent(vector<pll> &C, pll p) {
    auto gao = [&](int s) {
        return cyc_tsearch(SZ(C), [&](int x, int y)
            { return ori(p, C[x], C[y]) == s; });
    };
    return pii(gao(1), gao(-1));
} // return (a, b), ori(p, C[a], C[b]) >= 0
```

8.8 Intersection of line and convex [157258]

```
int TangentDir(vector<pll> &C, pll dir) {
    return cyc_tsearch(SZ(C), [&](int a, int b) {
        return cross(dir, C[a]) > cross(dir, C[b]);
    });
}
#define cmpl(i) sign(cross(C[i] - a, b - a))
pii lineHull(pll a, pll b, vector<pll> &C) {
    int A = TangentDir(C, a - b);
    int B = TangentDir(C, b - a);
    int n = SZ(C);
    if (cmpl(A) < 0 || cmpl(B) > 0)
        return pii(-1, -1); // no collision
    auto gao = [&](int l, int r) {
        for (int t = l; (l + 1) % n != r; ) {
```

```

    int m = ((l + r + (l < r ? 0 : n)) / 2) % n;
    (cmpL(m) == cmpL(t) ? l : r) = m;
}
return (l + !cmpL(r)) % n;
};
pii res = pii(gao(B, A), gao(A, B)); // (i, j)
if (res.X == res.Y) // touching the corner i
    return pii(res.X, -1);
if (!cmpL(res.X) && !cmpL(res.Y)) // along side i, i+1
    switch ((res.X - res.Y + n + 1) % n) {
        case 0: return pii(res.X, res.X);
        case 2: return pii(res.Y, res.Y);
    }
/* crossing sides (i, i+1) and (j, j+1)
crossing corner i is treated as side (i, i+1)
returned
    in the same order as the line hits the convex */
return res;
} // convex cut: (r, l]

```

8.9 minMaxEnclosingRectangle* [337ee3]

```

const double INF = 1e18, qi = acos(-1) / 2 * 3;
pdd solve(vector<pll> &dots) {
#define diff(u, v) (dots[u] - dots[v])
#define vec(v) (dots[v] - dots[i])
    hull(dots);
    double Max = 0, Min = INF, deg;
    int n = SZ(dots);
    dots.pb(dots[0]);
    for (int i = 0, u = 1, r = 1, l = 1; i < n; i++) {
        pll nw = vec(i + 1);
        while (cross(nw, vec(u + 1)) > cross(nw, vec(u)))
            u = (u + 1) % n;
        while (dot(nw, vec(r + 1)) > dot(nw, vec(r)))
            r = (r + 1) % n;
        if (!l) l = (r + 1) % n;
        while (dot(nw, vec(l + 1)) < dot(nw, vec(l)))
            l = (l + 1) % n;
        Min = min(Min, (double)(dot(nw, vec(r)) -
            dot(nw, vec(l))) * cross(nw, vec(u)) / abs2(nw));
        deg = acos(dot(diff(r, l), vec(u)) / abs(diff(r, l)) / abs(vec(u)));
        deg = (qi - deg) / 2;
        Max = max(Max, abs(
            diff(r, l)) * abs(vec(u)) * sin(deg) * sin(deg));
    }
    return pdd(Min, Max);
}

```

8.10 VectorInPoly* [c6d0fa]

```

// ori
(a, b, c) >= 0, valid: "strict" angle from a-b to a-c
bool btwangle(pll a, pll b, pll c, pll p, int strict) {
    return ori(a, b, p) >= strict && ori(a, p, c) >= strict;
}
// whether vector
{cur, p} in counter-clockwise order prv, cur, nxt
bool
    inside(pll prv, pll cur, pll nxt, pll p, int strict) {
    if (ori(cur, nxt, prv) >= 0)
        return btwangle(cur, nxt, prv, p, strict);
    return !btwangle(cur, prv, nxt, p, !strict);
}

```

8.11 PolyUnion* [5c3185]

```

double rat(pll a, pll b) {
    return
        sign(b.X) ? (double)a.X / b.X : (double)a.Y / b.Y;
} // all poly. should be ccw
double polyUnion(vector<vector<pll>> &poly) {
    double res = 0;
    for (auto &p : poly)
        FOR (a, 0, SZ(p) - 1) {
            pll A = p[a], B = p[(a + 1) % SZ(p)];
            vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
            for (auto &q : poly) {
                if (&p == &q) continue;
                FOR (b, 0, SZ(q) - 1) {
                    pll C = q[b], D = q[(b + 1) % SZ(q)];
                    int sc = ori(A, B, C), sd = ori(A, B, D);
                    if (sc != sd && min(sc, sd) < 0) {
                        double sa = cross(
                            (D - C, A - C), sb = cross(D - C, B - C);
                        segs.emplace_back(
                            (sa / (sa - sb), sign(sc - sd)));
                    }
                }
            }
        }
    }

```

```

    if (!sc && !sd
        && &q < &p && sign(dot(B - A, D - C)) > 0) {
        segs.emplace_back(rat(C - A, B - A), 1);
        segs.emplace_back(rat(D - A, B - A), -1);
    }
}
sort(ALL(segs));
for (auto &s : segs) s.X = clamp(s.X, 0.0, 1.0);
double sum = 0;
int cnt = segs[0].Y;
FOR (j, 1, SZ(segs) - 1) {
    if (!cnt) sum += segs[j].X - segs[j - 1].X;
    cnt += segs[j].Y;
}
res += cross(A, B) * sum;
}
return res / 2;
}

```

8.12 Polar Angle Sort* [b20533]

```

int cmp(pll a, pll b, bool same = true) {
#define is_neg(k) (sign(k.Y) < 0 || (sign(k.Y) == 0 && sign(k.X) < 0))
    int A = is_neg(a), B = is_neg(b);
    if (A != B)
        return A < B;
    if (sign(cross(a, b)) == 0)
        return same ? abs2(a) < abs2(b) : -1;
    return sign(cross(a, b)) > 0;
}

```

8.13 Half plane intersection* [3753as]

```

pll area_pair(Line a, Line b)
{ return pll(cross(
    a.Y - a.X, b.X - a.X), cross(a.Y - a.X, b.Y - a.X)); }
bool isin(Line l0, Line l1, Line l2) {
    // Check inter(l1, l2) strictly in l0
    auto [a02X, a02Y] = area_pair(l0, l2);
    auto [a12X, a12Y] = area_pair(l1, l2);
    if (a12X - a12Y < 0) a12X *= -1, a12Y *= -1;
    return
        (__int128) a02Y * a12X - (__int128) a02X * a12Y > 0;
}
/* Having solution, check size > 2 */
/* --^-- Line.X --^-- Line.Y --^-- */
vector<Line> halfPlaneInter(vector<Line> arr) {
    sort(ALL(arr), [&](Line a, Line b) -> int {
        if (cmp(a.Y - a.X, b.Y - b.X, 0) != -1)
            return cmp(a.Y - a.X, b.Y - b.X, 0);
        return ori(a.X, a.Y, b.Y) < 0;
    });
    deque<Line> dq(1, arr[0]);
    auto pop_back = [&](int t, Line p) {
        while (SZ
            (dq) >= t && !isin(p, dq[SZ(dq) - 2], dq.back()))
            dq.pop_back();
    };
    auto pop_front = [&](int t, Line p) {
        while (SZ(dq) >= t && !isin(p, dq[0], dq[1]))
            dq.pop_front();
    };
    for (auto p : arr)
        if (cmp
            (dq.back().Y - dq.back().X, p.Y - p.X, 0) != -1)
            pop_back(2, p), pop_front(2, p), dq.pb(p);
        pop_back(3, dq[0]), pop_front(3, dq.back());
        return vector<Line>(ALL(dq));
}

```

8.14 HPI Alternative Form [a3cb20]

```

using i128 = __int128;
struct LN {
    ll a, b, c; // ax + by + c <= 0
    pll dir() const { return pll(a, b); }
    LN(ll ta, ll tb, ll tc) : a(ta), b(tb), c(tc) {}
    LN(pll p, pll q)
        : a((q - p).Y), b(-(q - p).X), c(cross(q, p)) {}
};
pdd intersect(LN A, LN B) {
    double c = cross(A.dir(), B.dir());
    i128 a = i128(A.c) * B.a - i128(B.c) * A.a;
    i128 b = i128(A.c) * B.b - i128(B.c) * A.b;
    return pdd(-b / c, a / c);
}
bool cov(LN l, LN A, LN B) {

```

```

i128 c = cross(A.dir(), B.dir());
i128 a = i128(A.c) * B.a - i128(B.c) * A.a;
i128 b = i128(A.c) * B.b - i128(B.c) * A.b;
return sign(a * l.b - b * l.a + c * l.c) * sign(c) >= 0;
}
bool operator<(LN a, LN b) {
    if (int
        c = cmp(a.dir(), b.dir(), false); c != -1) return c;
    return i128(abs(b.a)
        + abs(b.b)) * a.c > i128(abs(a.a) + abs(a.b)) * b.c;
}

```

8.15 RotatingSweepLine [b2b894]

```

void rotatingSweepLine(vector<pii> &ps) {
    int n = SZ(ps), m = 0;
    vector<int> id(n), pos(n);
    vector<pii> line(n * (n - 1));
    FOR (i, 0, n - 1)
        FOR (j, 0, n - 1)
            if (i != j) line[m++] = pii(i, j);
    sort(ALL(line), [&](pii a, pii b) {
        return cmp(ps[a.Y] - ps[a.X], ps[b.Y] - ps[b.X]);
    }); // cmp(): polar angle compare
    iota(ALL(id), 0);
    sort(ALL(id), [&](int a, int b) {
        if (ps[a].Y != ps[b].Y) return ps[a].Y < ps[b].Y;
        return ps[a] < ps[b];
    }); // initial order, since (1, 0) is the smallest
    FOR (i, 0, n - 1) pos[id[i]] = i;
    FOR (i, 0, m - 1) {
        auto l = line[i];
        // do something
        tie(pos[l.X], pos[l.Y], id[pos[l.X]], id[pos[
            l.Y]]) = make_tuple(pos[l.Y], pos[l.X], l.Y, l.X);
    }
}

```

8.16 Minimum Enclosing Circle* [488721]

```

pdd Minimum_Enclosing_Circle
    (vector<pdd> dots, double &r) {
    pdd cent;
    static mt19937 rng(random_device{}());
    shuffle(ALL(dots), rng);
    cent = dots[0], r = 0;
    FOR (i, 1, SZ(dots) - 1)
        if (abs(dots[i] - cent) > r) {
            cent = dots[i], r = 0;
            FOR (j, 0, i - 1)
                if (abs(dots[j] - cent) > r) {
                    cent = (dots[i] + dots[j]) / 2;
                    r = abs(dots[i] - cent);
                    FOR (k, 0, j - 1)
                        if (abs(dots[k] - cent) > r) {
                            cent =
                                circenter(dots[i], dots[j], dots[k]);
                            r = abs(dots[i] - cent);
                        }
                }
        }
    return cent;
}

```

8.17 Intersection of two circles* [f7a2fe]

```

bool CCInter(Cir &a, Cir &b, pdd &p1, pdd &p2) {
    pdd o1 = a.O, o2 = b.O;
    double r1
        = a.R, r2 = b.R, d2 = abs2(o1 - o2), d = sqrt(d2);
    if (d <
        max(r1, r2) - min(r1, r2) || d > r1 + r2) return 0;
    pdd u = (o1 + o2) *
        0.5 + (o1 - o2) * ((r2 * r2 - r1 * r1) / (2 * d2));
    double A = sqrt((r1 + r2 + d
        ) * (r1 - r2 + d) * (r1 + r2 - d) * (-r1 + r2 + d));
    pdd v = pdd(o1.Y - o2.Y, -o1.X + o2.X) * A / (2 * d2);
    p1 = u + v, p2 = u - v;
    return 1;
}

```

8.18 Intersection of polygon and circle* [3e1eac]

```

// Divides into multiple triangle, and sum up
const double PI=acos(-1);
double _area(pdd pa, pdd pb, double r){
    if(abs(pa)<abs(pb)) swap(pa, pb);
    if(abs(pb)<eps) return 0;
    double ar, h, theta;

```

```

double a=abs(pb),b=abs(pa),c=abs(pb-pa);
double cosB = dot(pb,pb-pa) / a / c, B = acos(cosB);
double cosC = dot(pa,pb) / a / b, C = acos(cosC);
if(a > r){
    ar = (C / 2) * r * r;
    h = a*b*sin(C)/c;
    if (h < r && B < PI / 2)
        ar -= (acos(h / r) * r * r -
            h * sqrt(r * r - h * h));
}
else if(b > r){
    theta = PI - B - asin(sin(B)/r*a);
    ar = .5 * a * r * sin(theta) +
        (C - theta) / 2 * r * r;
} else ar = .5 * sin(C) * a * b;
return ar;
}
double area_poly_circle
    (const vector<pdd> poly,const pdd &o,const double r){
    double ar = 0;
    FOR (i, 0, SZ(poly) - 1)
        ar += _area(poly[i] - o,
            poly[(i + 1) % SZ(poly)] - o, r) *
            ori(o, poly[i], poly[(i + 1) % SZ(poly)]);
    return fabs(ar);
}

```

8.19 Intersection of line and circle* [7a7c59]

```

vector<pdd> circleLine(pdd c, double r, pdd a, pdd b) {
    pdd p = a + (b - a) * dot(c - a, b - a) / abs2(b - a);
    double s = cross
        (b - a, c - a), h2 = r * r - s * s / abs2(b - a);
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    pdd h = (b - a) / abs(b - a) * sqrt(h2);
    return {p - h, p + h};
}

```

8.20 Tangent line of two circles [2f476e]

```

vector<
    Line> go( const Cir& c1 , const Cir& c2 , int sign1 ){
    // sign1 = 1 for outer tang, -1 for inter tang
    vector<Line> ret;
    double d_sq = abs2(c1.O - c2.O);
    if (sign(d_sq) == 0) return ret;
    double d = sqrt(d_sq);
    pdd v = (c2.O - c1.O) / d;
    double c = (c1.R - sign1 * c2.R) / d;
    if (c * c > 1) return ret;
    double h = sqrt(max(0.0, 1.0 - c * c));
    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
        pdd n = pdd(v.X * c - sign2 * h * v.Y,
            v.Y * c + sign2 * h * v.X);
        pdd p1 = c1.O + n * c1.R;
        pdd p2 = c2.O + n * (c2.R * sign1);
        if (sign(p1.X - p2.X) == 0 and
            sign(p1.Y - p2.Y) == 0)
            p2 = p1 + perp(c2.O - c1.O);
        ret.pb(Line(p1, p2));
    }
    return ret;
}

```

8.21 CircleCover* [190c6f]

```

const int N = 1021;
struct CircleCover {
    int C;
    Cir c[N];
    bool g[N][N], overlap[N][N];
    // Area[i] : area covered by at least i circles
    double Area[ N ];
    void init(int _C){ C = _C;}
    struct Teve {
        pdd p; double ang; int add;
        Teve() {}
        Teve(pdd
            _a, double _b, int _c):p(_a), ang(_b), add(_c){}
        bool operator<(const Teve &a)const
            {return ang < a.ang;}
    }eve[N * 2];
    // strict: x = 0, otherwise x = -1
    bool disjuct(Cir &a, Cir &b, int x)
    {return sign(abs(a.O - b.O) - a.R - b.R) > x;}
    bool contain(Cir &a, Cir &b, int x)
    {return sign(a.R - b.R - abs(a.O - b.O)) > x;}
    bool contain(int i, int j) {

```

```

/* c[j] is non-strictly in c[i]. */
return (sign(c[i].R - c[j].R) > 0 || (sign(c[i].R - c[j].R) == 0 && i < j)) && contain(c[i], c[j], -1);
}

void solve(){
    fill_n(Area, C + 2, 0);
    FOR (i, 0, C - 1)
        FOR (j, 0, C - 1) overlap[i][j] = contain(i, j);
    FOR (i, 0, C - 1)
        FOR (j, 0, C - 1)
            g[i][j] = !(overlap[i][j] || overlap[j][i] || disjunct(c[i], c[j], -1));
    FOR (i, 0, C - 1) {
        int E = 0, cnt = 1;
        FOR (j, 0, C - 1)
            if (j != i && overlap[j][i])
                ++cnt;
        FOR (j, 0, C - 1)
            if (i != j && g[i][j]) {
                pdd aa, bb;
                CCinter(c[i], c[j], aa, bb);
                double
                    A = atan2(aa.Y - c[i].O.Y, aa.X - c[i].O.X);
                double
                    B = atan2(bb.Y - c[i].O.Y, bb.X - c[i].O.X);
                eve[E++] = Teve(bb, B, 1), eve[E++] = Teve(aa, A, -1);
                if (B > A) ++cnt;
            }
        if (E == 0) Area[cnt] += pi * c[i].R * c[i].R;
        else {
            sort(eve, eve + E);
            eve[E] = eve[0];
            FOR (j, 0, E - 1) {
                cnt += eve[j].add;
                Area[cnt] += cross(eve[j].p, eve[j + 1].p) * .5;
                double theta = eve[j + 1].ang - eve[j].ang;
                if (theta < 0) theta += 2. * pi;
                Area[cnt] += (theta - sin(theta)) * c[i].R * c[i].R * .5;
            }
        }
    }
}
};

```

8.22 3Dpoint* [badbbd]

```

struct Point {
    double x, y, z;
    Point(double _x = 0,
           double _y = 0, double _z = 0): x(_x), y(_y), z(_z){}
    Point(pdd p) { x = p.X, y = p.Y, z = abs2(p); }
};

Point operator-(Point p1, Point p2)
{ return Point(p1.x - p2.x, p1.y - p2.y, p1.z - p2.z); }
Point operator+(Point p1, Point p2)
{ return Point(p1.x + p2.x, p1.y + p2.y, p1.z + p2.z); }
Point operator*(Point p1, double v)
{ return Point(p1.x * v, p1.y * v, p1.z * v); }
Point operator/(Point p1, double v)
{ return Point(p1.x / v, p1.y / v, p1.z / v); }
Point cross(Point p1, Point p2)
{ return Point(p1.y * p2.z - p1.z * p2.y, p1.x * p2.y - p1.y * p2.x); }
double dot(Point p1, Point p2)
{ return p1.x * p2.x + p1.y * p2.y + p1.z * p2.z; }
double abs(Point a)
{ return sqrt(dot(a, a)); }
Point cross3(Point a, Point b, Point c)
{ return cross(b - a, c - a); }
double area(Point a, Point b, Point c)
{ return abs(cross3(a, b, c)); }
double volume(Point a, Point b, Point c, Point d)
{ return dot(cross3(a, b, c), d - a); }
//Azimuthal
    angle (longitude) to x-axis in interval [-pi, pi]
double phi(Point p) { return atan2(p.y, p.x); }
//Zenith
    angle (latitude) to the z-axis in interval [0, pi]
double theta(Point
    p) { return atan2(sqrt(p.x * p.x + p.y * p.y), p.z); }
Point masscenter(Point a, Point b, Point c, Point d)
{ return (a + b + c + d) / 4; }
pdd proj(Point a, Point b, Point c, Point u) {
    // proj. u to the plane of a, b, and c
    Point e1 = b - a;
    Point e2 = c - a;
    e1 = e1 / abs(e1);

```

```

    e2 = e2 - e1 * dot(e2, e1);
    e2 = e2 / abs(e2);
    Point p = u - a;
    return pdd(dot(p, e1), dot(p, e2));
}
Point rotate_around(Point p, double angle, Point axis) {
    double s = sin(angle), c = cos(angle);
    Point u = axis / abs(axis);
    return
        u * dot(u, p) * (1 - c) + p * c + cross(u, p) * s;
}

```

8.23 Convexhull3D* [ae9cc8]

```

struct convex_hull_3D {
    struct Face {
        int a, b, c;
        Face(int ta, int tb, int tc): a(ta), b(tb), c(tc) {}
    }; // return the faces with pt indexes
    vector<Face> res;
    vector<Point> P;
    convex_hull_3D(const vector<Point> &P): res(), P(_P) {
        // all points coplanar case will WA, O(n^2)
        int n = SZ(P);
        if (n <= 2) return; // be careful about edge case
        // ensure first 4 points are not coplanar
        swap(P[1], *find_if(ALL(P), [&](auto p) { return sign(abs2(P[0] - p)) != 0; }));
        swap(P[2], *find_if(ALL(P), [&](auto p) { return sign(abs2(cross3(p, P[0], P[1]))) != 0; }));
        swap(P[3], *find_if(ALL(P), [&](auto p) { return sign(volume(P[0], P[1], P[2], p)) != 0; }));
        vector<vector<int>> flag(n, vector<int>(n));
        res.emplace_back(0, 1, 2); res.emplace_back(2, 1, 0);
        FOR (i, 3, n - 1) {
            vector<Face> next;
            for (auto f : res) {
                int d = sign(volume(P[f.a], P[f.b], P[f.c], P[i]));
                if (d <= 0) next.pb(f);
                int ff = (d > 0) - (d < 0);
                flag[f.a][f.b] = flag[f.b][f.c] = flag[f.c][f.a] = ff;
            }
            for (auto f : res) {
                auto add = [&](int x, int y) {
                    if (flag[x][y] > 0 && flag[y][x] <= 0)
                        next.emplace_back(x, y, i);
                };
                add(f.a, f.b);
                add(f.b, f.c);
                add(f.c, f.a);
            }
            res = next;
        }
    }
    bool same(Face s, Face t) {
        if (sign(volume(P[s.a], P[s.b], P[s.c], P[t.a])) != 0) return 0;
        if (sign(volume(P[s.a], P[s.b], P[s.c], P[t.b])) != 0) return 0;
        if (sign(volume(P[s.a], P[s.b], P[s.c], P[t.c])) != 0) return 0;
        return 1;
    }
    int polygon_face_num() {
        int ans = 0;
        FOR (i, 0, SZ(res) - 1)
            ans += none_of(res.begin(), res.begin() + i, [&](Face g) { return same(res[i], g); });
        return ans;
    }
    double get_volume() {
        double ans = 0;
        for (auto f : res)
            ans += volume(Point(0, 0, 0), P[f.a], P[f.b], P[f.c]);
        return fabs(ans / 6);
    }
    double get_dis(Point p, Face f) {
        Point p1 = P[f.a], p2 = P[f.b], p3 = P[f.c];
        double a = (p2.y - p1.y) * (p3.z - p1.z) - (p2.z - p1.z) * (p3.y - p1.y);
        double b = (p2.z - p1.z) * (p3.x - p1.x) - (p2.x - p1.x) * (p3.z - p1.z);
        double c = (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);
        double d = 0 - (a * p1.x + b * p1.y + c * p1.z);
        return fabs(a * p.x + b * p.y + c * p.z + d) / sqrt(a * a + b * b + c * c);
    }
}

```

```
};
// n^2 delaunay: facets with negative z normal of
// convexhull of (x, y, x^2 + y^2), use a pseudo-point
// (0, 0, inf) to avoid degenerate case
```

8.24 DelaunayTriangulation* [4562b4]

```
/* Delaunay Triangulation:
Given a sets of points on 2D plane, find a
triangulation such that no points will strictly
inside circumcircle of any triangle. */
struct Edge {
    int id; // oidx[id]
    list<Edge>::iterator twin;
    Edge(int _id = 0):id(_id) {}
};

struct Delaunay { // 0-base
    int n, oidx[N];
    list<Edge> head[N]; // result udir. graph
    pll p[N];
    void init(int _n, pll _p[]) {
        n = _n, iota(oidx, oidx + n, 0);
        FOR (i, 0, n - 1) head[i].clear();
        sort(oidx, oidx + n, [&](int a, int b)
        { return _p[a] < _p[b]; });
        FOR (i, 0, n - 1) p[i] = _p[oidx[i]];
        divide(0, n - 1);
    }
    void addEdge(int u, int v) {
        head[u].push_front(Edge(v));
        head[v].push_front(Edge(u));
        head[u].begin()->twin = head[v].begin();
        head[v].begin()->twin = head[u].begin();
    }
    void divide(int l, int r) {
        if (l == r) return;
        if (l + 1 == r) return addEdge(l, l + 1);
        int mid = (l + r) >> 1, nw[2] = {l, r};
        divide(l, mid), divide(mid + 1, r);
        auto gao = [&](int t) {
            pll pt[2] = {p[nw[0]], p[nw[1]]};
            for (auto it : head[nw[t]]) {
                int v = ori(pt[1], pt[0], p[it.id]);
                if (v > 0 || (v == 0 && abs2
                    (pt[t ^ 1] - p[it.id]) < abs2(pt[1] - pt[0])))
                    return nw[t] = it.id, true;
            }
            return false;
        };
        while (gao(0) || gao(1));
        addEdge(nw[0], nw[1]); // add tangent
        while (true) {
            pll pt[2] = {p[nw[0]], p[nw[1]]};
            int ch = -1, sd = 0;
            FOR (t, 0, 1)
                for (auto it : head[nw[t]])
                    if (ori(pt[0], pt[1], p[it.id]) > 0 &&
                        (ch == -1 ||
                         in_cc({pt[0], pt[1], p[ch]}, p[it.id])))
                        ch = it.id, sd = t;
            if (ch == -1) break; // upper common tangent
            for (auto it = head
                [nw[sd]].begin(); it != head[nw[sd]].end(); )
                if (seg_strict_intersect
                    (pt[sd], p[it->id], pt[sd ^ 1], p[ch]))
                    head[it->id]
                        .erase(it->twin), head[nw[sd]].erase(it++);
            else ++it;
            nw[sd] = ch, addEdge(nw[0], nw[1]);
        }
    }
}; tool;
```

8.25 Triangulation Voronoi* [6ddd07]

```
// all coord. is
even, you may want to call halfPlaneInter after then
vector<vector<Line>> vec;
void build_voronoi_line(int n, pll *arr) {
    tool.init(n, arr); // Delaunay
    vec.clear(), vec.resize(n);
    FOR (i, 0, n - 1)
        for (auto e : tool.head[i]) {
            int u = tool.oidx[i], v = tool.oidx[e.id];
            pll m = (arr
                [v] + arr[u]) / 2LL, d = perp(arr[v] - arr[u]);
            vec[u].pb(Line(m, m + d));
        }
}
```

8.26 Minkowski Sum* [85a883]

```
vector<pll> Minkowski
(vector<pll> A, vector<pll> B) { // |A|,|B|>=3
    hull(A), hull(B);
    vector<pll> C(1, A[0] + B[0]), s1, s2;
    FOR (i, 0, SZ(A) - 1)
        s1.pb(A[(i + 1) % SZ(A)] - A[i]);
    FOR (i, 0, SZ(B) - 1)
        s2.pb(B[(i + 1) % SZ(B)] - B[i]);
    for (int i = 0, j = 0; i < SZ(A) || j < SZ(B);)
        if (j >=
            SZ(B) || (i < SZ(A) && cross(s1[i], s2[j]) >= 0))
            C.pb(B[j % SZ(B)] + A[i++]);
        else
            C.pb(A[i % SZ(A)] + B[j++]);
    return hull(C), C;
}
```

8.27 Closest Pair [6f81f1]

```
// closest pair; only needs pll + X/Y
pair<pll, pll> closest_pair(vector<pll> v) {
    sort(ALL(v), [](pll a, pll b) { return a.Y < b.Y; });
    pair<ll, pair<pll, pll>> r{LLONG_MAX, {}};
    set<pll> s;
    int j = 0;
    for (pll p : v) {
        ll d = 1 + (ll)sqrt(r.F);
        while (v[j].Y <= p.Y - d) s.erase(v[j++]);
        auto lo = s.lower_bound(pll(p.X - d, p.Y));
        auto hi = s.upper_bound(pll(p.X + d, p.Y));
        for (; lo != hi; lo++) {
            ll dx = lo->X - p.X, dy = lo->Y - p.Y;
            r = min(r, {dx * dx + dy * dy, {lo, p}});
        }
        s.insert(p);
    }
    return r.S; // r.F is the squared distance
}
```

9 Else

9.1 Cyclic Ternary Search* [9017cc]

```
/* bool pred(int a, int b);
f(0) ~ f(n - 1) is a cyclic-shift U-function
return idx s.t. pred(x, idx) is false forall x*/
int cyc_tsearch(int n, auto pred) {
    if (n == 1) return 0;
    int l = 0, r = n; bool rv = pred(1, 0);
    while (r - l > 1) {
        int m = (l + r) / 2;
        if (pred(0, m) ? rv : pred(m, (m + 1) % n)) r = m;
        else l = m;
    }
    return pred(l, r % n) ? l : r % n;
}
```

9.2 Mo's Algorithm(With modification) [dcb6b7]

```
/*
Mo's Algorithm With modification
Block: N^{2/3}, Complexity: N^{5/3}
*/
struct Query {
    int L, R, LBid, RBid, T;
    Query(int l, int r, int t):
        L(l), R(r), LBid(l / blk), RBid(r / blk), T(t) {}
    bool operator<(const Query &q) const {
        if (LBid != q.LBid) return LBid < q.LBid;
        if (RBid != q.RBid) return RBid < q.RBid;
        return T < b.T;
    }
};
void solve(vector<Query> query) {
    sort(query.begin(), query.end());
    int L = 0, R = 0, T = -1;
    for (auto q : query) {
        while (T < q.T) addTime(L, R, ++T); // TODO
        while (T > q.T) subTime(L, R, T--); // TODO
        while (R < q.R) add(arr[++R]); // TODO
        while (L > q.L) add(arr[--L]); // TODO
        while (R > q.R) sub(arr[R--]); // TODO
        while (L < q.L) sub(arr[L++]); // TODO
        // answer query
    }
}
```

9.3 Mo's Algorithm On Tree [29ff55]

```

/*
Mo's Algorithm On Tree
Preprocess:
1) LCA
2) dfs with in[u] = dft++, out[u] = dft++
3) ord[in[u]] = ord[out[u]] = u
4) bitset<MAXN> inset
*/
struct Query {
    int L, R, LBid, lca;
    Query(int u, int v) {
        int c = LCA(u, v);
        if (c == u || c == v)
            q.lca = -1, q.L = out[c ^ u ^ v], q.R = out[c];
        else if (out[u] < in[v])
            q.lca = c, q.L = out[u], q.R = in[v];
        else
            q.lca = c, q.L = out[v], q.R = in[u];
        q.Lid = q.L / blk;
    }
}
bool operator<(const Query &q) const {
    if (LBid != q.LBid) return LBid < q.LBid;
    return R < q.R;
}
};
void flip(int x) {
    if (inset[x]) sub(arr[x]); // TODO
    else add(arr[x]); // TODO
    inset[x] = ~inset[x];
}
void solve(vector<Query> query) {
    sort(query.begin(), query.end());
    int L = 0, R = 0;
    for (auto q : query) {
        while (R < q.R) flip(ord[++R]);
        while (L > q.L) flip(ord[--L]);
        while (R > q.R) flip(ord[R--]);
        while (L < q.L) flip(ord[L++]);
        if (~q.lca) add(arr[q.lca]);
        // answer query
        if (~q.lca) sub(arr[q.lca]);
    }
}

```

9.4 Additional Mo's Algorithm Trick

- Mo's Algorithm With Addition Only
 - Sort queries same as the normal Mo's algorithm.
 - For each query $[l, r]$:
 - If $l/blk = r/blk$, brute-force.
 - If $l/blk \neq r/blk$, initialize $curL := (l/blk + 1) \cdot blk$, $curR := curL - 1$
 - If $r > curR$, increase $curR$
 - decrease $curL$ to fit l , and then undo after answering
- Mo's Algorithm With Offline Second Time
 - Require: Changing answer $\equiv$ adding $f([l, r], r+1)$.
 - Require: $f([l, r], r+1) = f([l, r], r+1) - f([l, l], r+1)$.
 - Part1: Answer all $f([l, r], r+1)$ first.
 - Part2: Store $curR \rightarrow R$ for $curL$ (reduce the space to $O(N)$), and then answer them by the second offline algorithm.
 - Note: You must do the above symmetrically for the left boundaries.

9.5 Hilbert Curve [1274a3]

```

ll hilbert(int n, int x, int y) {
    ll res = 0;
    for (int s = n / 2; s; s >>= 1) {
        int rx = (x & s) > 0;
        int ry = (y & s) > 0;
        res += s * 1ll * s * ((3 * rx) ^ ry);
        if (ry == 0) {
            if (rx == 1) x = s - 1 - x, y = s - 1 - y;
            swap(x, y);
        }
    }
    return res;
} // n = 2^k

```

9.6 DynamicConvexTrick* [673ffd]

```

// only works for integer coordinates!! maintain max
struct Line {
    mutable ll a, b, p;
    bool operator
        <(const Line &rhs) const { return a < rhs.a; }
    bool operator<(ll x) const { return p < x; }
};
struct DynamicHull : multiset<Line, less<>> {
    static const ll kInf = 1e18;
    ll Div(ll
        a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }
}

```

```

bool isect(iterator x, iterator y) {
    if (y == end()) { x->p = kInf; return 0; }
    if (x->a == y->a) x->p = x->b > y->b ? kInf : -kInf;
    else x->p = Div(y->b - x->b, x->a - y->a);
    return x->p >= y->p;
}
void addline(ll a, ll b) {
    auto z = insert({a, b, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x !=
        begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) !=
        begin() && (--x)->p >= y->p) isect(x, erase(y));
}
ll query(ll x) {
    auto l = *lower_bound(x);
    return l.a * x + l.b;
}
};

```

9.7 AllLCS* [84c2d5]

```

struct AllLCS { // 0-base
    string t;
    vector<int> h;
    AllLCS(const string &t) : t(t), h(t.size()) {
        iota(h.begin(), h.end(), 0);
    }
    void push(char c) { // append c to the other string
        int v = -1;
        FOR (i, 0, (int)t.size() - 1)
            if (t[i] == c || h[i] < v) swap(h[i], v);
        // after pushing s[0, a], for every b <= c:
        // LCS(s[0, a], t[b, c]) =
        // c - b + 1 - #{i <= c : h[i] >= b}
        // h[i] might become -1 !!
    }
};

```

9.8 DLX* [eef767]

```

#define TRAV(i, link, start) \
    for (int i = link[start]; i != start; i = link[i])
template <bool E> // E: Exact, NN: num of 1s, RR: num
// of rows
struct DLX {
    int lt[NN], rg[NN], up[NN], dn[NN], rw[NN], cl[NN],
        bt[NN], s[NN], head, sz, ans;
    int rows, columns;
    bool vis[NN];
    bitset<RR> sol, cur; // not sure
    void remove(int c) {
        if (E) lt[rg[c]] = lt[c], rg[lt[c]] = rg[c];
        TRAV(i, dn, c) {
            if (E) {
                TRAV(j, rg, i)
                    up[dn[j]] = up[j], dn[up[j]] = dn[j],
                    --s[cl[j]];
            } else {
                lt[rg[i]] = lt[i], rg[lt[i]] = rg[i];
            }
        }
    }
    void restore(int c) {
        TRAV(i, up, c) {
            if (E) {
                TRAV(j, lt, i)
                    ++s[cl[j]], up[dn[j]] = j, dn[up[j]] = j;
            } else {
                lt[rg[i]] = rg[lt[i]] = i;
            }
        }
        if (E) lt[rg[c]] = c, rg[lt[c]] = c;
    }
    void init(int c) {
        rows = 0, columns = c;
        FOR (i, 0, c - 1) {
            up[i] = dn[i] = bt[i] = i;
            lt[i] = i == 0 ? c : i - 1;
            rg[i] = i == c - 1 ? c : i + 1;
            s[i] = 0;
        }
        rg[c] = 0, lt[c] = c - 1;
        up[c] = dn[c] = -1;
        head = c, sz = c + 1;
    }
    void insert(const vector<int> &col) {
        if (col.empty()) return;
        int f = sz;
    }
}

```

```

FOR (i, 0, (int)col.size() - 1) {
    int c = col[i], v = sz++;
    dn[bt[c]] = v;
    up[v] = bt[c], bt[c] = v;
    rg[v] = (i + 1 == (int)col.size() ? f : v + 1);
    rw[v] = rows, cl[v] = c;
    ++s[c];
    if (i > 0) lt[v] = v - 1;
}
++rows, lt[f] = sz - 1;
}
int h() {
    int ret = 0;
    fill_n(vis, sz, false);
    TRAV(x, rg, head) {
        if (vis[x]) continue;
        vis[x] = true, ++ret;
        TRAV(i, dn, x) TRAV(j, rg, i) vis[cl[j]] = true;
    }
    return ret;
}
void dfs(int dep) {
    if (dep + (E ? 0 : h()) >= ans) return;
    if (rg[head] == head)
        return sol = cur, ans = dep, void();
    if (dn[rg[head]] == rg[head]) return;
    int w = rg[head];
    TRAV(x, rg, head) if (s[x] < s[w]) w = x;
    if (E) remove(w);
    TRAV(i, dn, w) {
        if (!E) remove(i);
        TRAV(j, rg, i) remove(E ? cl[j] : j);
        cur.set(rw[i]), dfs(dep + 1), cur.reset(rw[i]);
        TRAV(j, lt, i) restore(E ? cl[j] : j);
        if (!E) restore(i);
    }
    if (E) restore(w);
}
int solve() {
    FOR (i, 0, columns - 1)
        dn[bt[i]] = i, up[i] = bt[i];
    ans = 1e9, sol.reset(), dfs(0);
    return ans;
}
};

```

9.9 Matroid Intersection

Start from $S = \emptyset$. In each iteration, let

- $Y_1 = \{x \notin S \mid S \cup \{x\} \in I_1\}$
- $Y_2 = \{x \notin S \mid S \cup \{x\} \in I_2\}$

If there exists $x \in Y_1 \cap Y_2$, insert x into S . Otherwise for each $x \in S, y \notin S$, create edges

- $x \rightarrow y$ if $S - \{x\} \cup \{y\} \in I_1$.
- $y \rightarrow x$ if $S - \{x\} \cup \{y\} \in I_2$.

Find a *shortest* path (with BFS) starting from a vertex in Y_1 and ending at a vertex in Y_2 which doesn't pass through any other vertices in Y_2 , and alternate the path. The size of S will be incremented by 1 in each iteration. For the weighted case, assign weight $w(x)$ to vertex x if $x \in S$ and $-w(x)$ if $x \notin S$. Find the path with the minimum number of edges among all minimum length paths and alternate it.

9.10 Matroid Intersection [63f9dd]

```

// max common independent set of two matroids on
// {0..n-1}. Each matroid provides
// void build(const vector<int> &s) // s[i]: i in S?
// bool ind(int x, int y) // S - x + y independent?
// // // x = -1 means just S + y
// O(r) rounds, each O(n^2) oracle calls. Weighted ver:
// replace the BFS with Bellman-Ford minimising
// (sum of w(x), x in S / -w(y), y not in S; #edges).
template <class M1, class M2>
vector<int> matroid_isect(int n, M1 &m1, M2 &m2) {
    vector<int> sel(n, 0);
    while (1) {
        m1.build(sel), m2.build(sel);
        vector<int> pre(n, -2), q;
        FOR (y, 0, n - 1)
            if (!sel[y] && m1.ind(-1, y))
                pre[y] = -1, q.pb(y);
        int t = -1;
        for (int h = 0; h < (int)q.size(); h++) {
            int u = q[h];
            if (!sel[u] && m2.ind(-1, u)) {
                t = u;
                break;
            }
        }
        FOR (v, 0, n - 1)
            if (pre[v] == -2)
                if (sel[u] ? !sel[v] && m1.ind(u, v)

```

```

                : sel[v] && m2.ind(v, u))
                    pre[v] = u, q.pb(v);
            }
            if (t < 0) return sel;
            for (int v = t; v != -1; v = pre[v]) sel[v] ^= 1;
        }
    }
    struct ColorMat { // <= cap[c] chosen of colour c
        vector<int> col, cap, cnt;
        ColorMat(vector<int> col, vector<int> cap)
            : col(col), cap(cap) {}
        void build(const vector<int> &s) {
            cnt.assign(cap.size(), 0);
            FOR (i, 0, (int)s.size() - 1)
                if (s[i]) cnt[col[i]]++;
        }
        bool ind(int x, int y) {
            int c = cnt[col[y]];
            if (x >= 0 && col[x] == col[y]) c--;
            return c < cap[col[y]];
        }
    };
    struct GraphMat { // chosen edges form a forest
        int n;
        vector<pair<int, int>> e;
        vector<int> sel, f;
        GraphMat(int n, vector<pair<int, int>> e)
            : n(n), e(e), f(n) {}
        void build(const vector<int> &s) { sel = s; }
        int fd(int a) {
            while (f[a] != a) a = f[a] = f[f[a]];
            return a;
        }
        bool ind(int x, int y) {
            iota(f.begin(), f.end(), 0);
            FOR (i, 0, (int)e.size() - 1)
                if (sel[i] && i != x) f[fd(e[i].F)] = fd(e[i].S);
            return fd(e[y].F) != fd(e[y].S);
        }
    };
};

```

9.11 AdaptiveSimpson* [b98c97]

```

template <typename Func, typename d = double>
struct Simpson {
    using pdd = pair<d, d>;
    Func f;
    pdd mix(pdd l, pdd r, optional<d> fm = {}) {
        d h = (r.F - l.F) / 2, v = fm.value_or(f(l.F + h));
        return {v, h / 3 * (l.S + 4 * v + r.S)};
    }
    d eval(pdd l, pdd r, d fm, d eps) {
        pdd m((l.F + r.F) / 2, fm);
        d s = mix(l, r, fm).S;
        auto [flm, sl] = mix(l, m);
        auto [fmr, sr] = mix(m, r);
        d delta = sl + sr - s;
        if (abs(delta) <= 15 * eps)
            return sl + sr + delta / 15;
        return eval(l, m, flm, eps / 2) +
            eval(m, r, fmr, eps / 2);
    }
    d eval(d l, d r, d eps) {
        return eval(
            {l, f(l)}, {r, f(r)}, f((l + r) / 2), eps);
    }
    d eval2(d l, d r, d eps, int k = 997) {
        d h = (r - l) / k, s = 0;
        for (int i = 0; i < k; i++, l += h)
            s += eval(l, l + h, eps / k);
        return s;
    }
};
template <typename Func>
Simpson<Func> make_simpson(Func f) {
    return {f};
}

```

9.12 Simulated Annealing [a6e293]

```

double factor = 100000;
const int base = 1e9; // remember to run ~ 10 times
FOR (it, 1, 100000) {
    // ans: answer, nw: current value, rnd(): mt19937
    // rnd()
    if (exp(-(nw - ans) / factor) >=
        (double)(rnd() % base) / base)
        ans = nw;
    factor *= 0.99995;
}

```

}

9.13 Tree Hash* [6e1149]

```
typedef unsigned long long ull; // need G, seed
ull seed, h[N]; // h[u]: hash of u's subtree
ull shift(ull x) {
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    return x;
}
ull dfs(int u, int f) {
    ull sum = seed;
    for (int i : G[u])
        if (i != f)
            sum += shift(dfs(i, u));
    return h[u] = sum;
}
```

9.14 Binary Search On Fraction* [765c5a]

```
struct Q {
    ll p, q;
    Q go(Q b, ll d) { return {p + b.p*d, q + b.q*d}; }
};
bool pred(Q);
// returns smallest p/q in [lo, hi] such that
// pred(p/q) is true, and 0 <= p, q <= N
Q frac_bs(ll N) {
    Q lo{0, 1}, hi{1, 0};
    if (pred(lo)) return lo;
    assert(pred(hi));
    bool dir = 1, L = 1, H = 1;
    for (; L || H; dir = !dir) {
        ll len = 0, step = 1;
        for (int t = 0; t < 2 && (t ? step/=2 : step*=2);)
            if (Q mid = hi.go(lo, len + step);
                mid.p > N || mid.q > N || dir ^ pred(mid))
                t++;
            else len += step;
        swap(lo, hi = hi.go(lo, len));
        (dir ? L : H) = !!len;
    }
    return dir ? hi : lo;
}
```

9.15 1D1D [590661]

```
typedef pair<int, int> pii;
// cost(i, j): cost of transit from i to j
// search(i, j): last point when i is better in row i,
// j, need to check whether cost(i, i) is better defined
int cost(int i, int j);
int search(int i, int j, int L, int R = N);
void concave1D1D() {
    vector<int> dp(N + 1, 0);
    vector<pii> st; // (idx, right end h_idx)
    st.emplace_back(pii(0, N + 1));
    FOR (j, 1, N) {
        while (st.back().S < j) st.pop_back();
        int i = st.back().F;
        dp[j] = dp[i] + cost(i, j);
        while (
            !st.empty() && search(j, i, j) >= st.back().S) {
            st.pop_back();
            i = st.back().F;
        }
        st.emplace_back(pii(j, search(j, i, j)));
    }
}
void convex1D1D() {
    vector<int> dp(N + 1, 0);
    deque<pii> dq; // (idx, right end)
    dq.emplace_back(pii(0, N + 1));
    FOR (j, 1, N) {
        while (dq.back().S < j) dq.pop_back();
        int i = dq.back().F;
        dp[j] = dp[i] + cost(i, j);
        while (dq.size() >= 2 &&
            search(i, j, j) <= dq.end()[-2].S) {
            dq.pop_back();
            i = dq.back().F;
        }
        dq.back().S = search(i, j, j);
        dq.emplace_back(pii(j, N + 1));
    }
}
```

9.16 SMAWK [4f281d]

```
long long query(int l, int r);
struct SMAWK {
    // Condition:
    // If M[1][0] < M[1][1] then M[0][0] < M[0][1]
    // If M[1][0] == M[1][1] then M[0][0] <= M[0][1]
    // For all i, find r_i s.t. M[i][r_i] is maximum //
    // minimum.
    int ans[N], tmp[N];
    void interpolate(vector<int> l, vector<int> r) {
        int n = l.size(), m = r.size();
        vector<int> nl;
        for (int i = 1; i < n; i += 2) { nl.pb(l[i]); }
        run(nl, r);
        for (int i = 1, j = 0; i < n; i += 2) {
            while (j < m && r[j] < ans[l[i]]) j++;
            assert(j < m && ans[l[i]] == r[j]);
            tmp[l[i]] = j;
        }
        for (int i = 0; i < n; i += 2) {
            int curl = 0, curr = m - 1;
            if (i) curl = tmp[l[i - 1]];
            if (i + 1 < n) curr = tmp[l[i + 1]];
            long long res = query(l[i], r[curl]);
            ans[l[i]] = r[curl];
            FOR (j, curl + 1, curr) {
                ll nxt = query(l[i], r[j]);
                if (res < nxt) res = nxt, ans[l[i]] = r[j];
            }
        }
    }
    void reduce(vector<int> l, vector<int> r) {
        int n = l.size(), m = r.size();
        vector<int> nr;
        for (int j : r) {
            while (!nr.empty()) {
                int i = nr.size() - 1;
                if (query(l[i], nr.back()) <= query(l[i], j))
                    nr.pop_back();
                else break;
            }
            if (nr.size() < n) nr.pb(j);
        }
        run(l, nr);
    }
    void run(vector<int> l, vector<int> r) {
        int n = l.size(), m = r.size();
        if (max(n, m) <= 2) {
            for (int i : l) {
                ans[i] = r[0];
                if (m > 1) {
                    if (query(i, r[0]) < query(i, r[1]))
                        ans[i] = r[1];
                }
            }
        } else if (n >= m) {
            interpolate(l, r);
        } else {
            reduce(l, r);
        }
    }
};
```

9.17 Min Plus Convolution* [5d8ac1]

```
// a is convex a[i+1]-a[i] <= a[i+2]-a[i+1]
vector<int> min_plus_convolution(
    vector<int> &a, vector<int> &b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<int> c(n + m - 1, INF);
    auto dc = [&](auto Y, int l, int r, int jl, int jr) {
        if (l > r) return;
        int mid = (l + r) / 2, from = -1, &best = c[mid];
        FOR (j, jl, jr)
            if (int i = mid - j; i >= 0 && i < n)
                if (best > a[i] + b[j])
                    best = a[i] + b[j], from = j;
        Y(Y, l, mid - 1, jl, from),
        Y(Y, mid + 1, r, from, jr);
    };
    return dc(dc, 0, n - 1 + m - 1, 0, m - 1), c;
}
```

9.18 Bitset LCS [83b00e]

```
cin >> n >> m;
for (int i = 1, x; i <= n; i++) cin >> x, p[x].set(i);
for (int i = 1, x; i <= m; i++) {
    cin >> x, (g = f) |= p[x];
    f.shiftLeftByOne(), f.set(0);
}
```

```
((f = g - f) ^= g) &= g;
}
cout << f.count() << '\n';
```

9.19 Python

```
from decimal import *
setcontext(Context(
    prec=MAX_PREC, Emax=MAX_EMAX, rounding=ROUND_HALF_UP))
print(Decimal(input()) * Decimal(input()))
from fractions import Fraction
Fraction('3.14159').limit_denominator(10).numerator # 22
```